



南開大學
Nankai University

计算机学院
实验报告

推荐系统评分预测项目

姓名：蔡子轩 学号：2310984

姓名：陈宇昕 学号：2310675

专业：计算机科学与技术

2026 年 6 月 29 日

目录

项目摘要	3
1 数据集基本统计信息	3
1.1 数据格式与任务建模	3
1.2 训练集规模与评分矩阵稀疏性	3
1.3 用户与物品评分分布分析	4
1.4 评分取值分布与预测边界	5
1.5 测试集冷启动情况与输出要求	5
2 Baseline 算法详细说明	6
2.1 算法思想	6
2.2 全局平均分	6
2.3 程序结构与核心参数	7
2.4 训练数据组织	7
2.5 用户偏置	8
2.6 物品偏置	8
2.7 交替迭代训练过程	8
2.8 预测与边界处理	10
3 ItemCF 算法详细说明	10
3.1 算法思想	10
3.2 程序结构	11
3.3 残差矩阵构建	11
3.4 物品相似度计算	12
3.5 相似邻居保留	13
3.6 基于相似邻居的评分预测	14
3.7 回退机制与边界处理	15
4 Matrix Factorization 算法详细说明	15
4.1 算法思想	15
4.2 程序结构	17
4.3 编号映射与数据准备	17
4.4 参数初始化	18
4.5 训练目标与 SGD 更新	19
4.6 预测与冷启动处理	21
4.7 核心结构占用估计	21
5 UserCF 算法详细说明	22
5.1 算法思想	22
5.2 程序实现	22

6	Ensemble 融合算法详细说明	23
6.1	算法思想	23
6.2	程序结构	23
6.3	子模型训练过程	24
6.4	融合预测过程	24
6.5	冷启动与核心结构占用	25
7	实验设计与结果分析	25
7.1	验证方法与评价指标	25
7.2	基础模型主实验	28
7.3	参数调优实验	30
7.3.1	Baseline 迭代轮数与正则参数	30
7.3.2	ItemCF 参数调优	30
7.3.3	Matrix Factorization 参数调优	35
7.3.4	融合权重调优	38
7.4	优化模型综合对比	40
7.5	多随机种子稳健性验证	41
7.6	最终模型选择与结果文件检查	42
8	总结	43

项目摘要

本实验完成推荐系统显式评分预测任务，在只使用 `Train.txt` 的前提下，预测 `Test.txt` 中 9982 个用户-物品对的未知评分。训练集中共有 598 个用户、9077 个物品和 90854 条评分记录，评分矩阵密度仅为 1.6738%，同时测试集中存在未见用户和未见物品，因此模型设计需要同时考虑稀疏性、冷启动回退和输出格式约束。

在算法方面，实验依次实现并比较了 Baseline、ItemCF、UserCF、Matrix Factorization 和 Ensemble 融合模型。Baseline 用于刻画全局平均分、用户偏置和物品偏置；ItemCF 与 UserCF 分别从物品相似性和用户相似性角度修正 Baseline 残差；矩阵分解使用低维隐向量学习潜在用户-物品交互；融合模型则利用不同推荐器误差结构的互补性进行线性加权。

实验采用 `Train.txt` 内部 5 折交叉验证和多随机种子验证进行模型选择。最终，ItemCF、Matrix Factorization 与 UserCF 的加权融合模型在 5 个随机种子构成的 25 个验证折上取得平均 RMSE 16.859807，优于原二模型融合的 16.980670，并在 25 个验证折中全部胜出。最终提交文件由该融合模型在完整训练集上重新训练后生成，经格式检查确认与 `Test.txt` 的用户块、物品数量和物品顺序完全一致，预测评分均位于 10 到 100 的合法范围内。

1 数据集基本统计信息

1.1 数据格式与任务建模

本实验面向推荐系统中的显式评分预测问题。数据集由训练集、测试集、结果格式文件和数据格式说明文件组成，其中 `Train.txt` 保存已知用户评分，`Test.txt` 给出需要预测的用户-物品对，`ResultForm.txt` 用于说明提交结果的组织形式，`DataFormatExplanation.txt` 说明原始文本数据的块状结构。

`Train.txt` 中每个用户块以 `user_id|rating_count` 开始，后续 `rating_count` 行分别记录该用户评分过的物品及其评分，格式为 `item_id score`。例如，一个用户块可以被理解为某个用户在历史行为中对若干物品给出的显式反馈。`Test.txt` 的块头同样表示用户编号和待预测物品数量，但每个物品行只包含 `item_id`，不包含真实评分。

根据该格式，训练数据可建模为评分集合

$$R = \{(u, i, r_{ui})\} \quad (1)$$

其中 u 表示用户， i 表示物品， r_{ui} 表示用户 u 对物品 i 的真实评分。测试数据则建模为待预测集合

$$T = \{(u, i)\} \quad (2)$$

实验目标是在只使用 R 的前提下学习评分函数 $\hat{r}_{ui} = f(u, i)$ ，并对 T 中全部用户-物品对输出预测评分。由于测试集没有真实评分，`Test.txt` 不能用于计算测试集 RMSE，也不能参与模型训练、参数调节或模型选择。因此，报告中的误差指标均在 `Train.txt` 内部划分出的验证集上计算。

1.2 训练集规模与评分矩阵稀疏性

读取 `Train.txt` 后，程序首先统计用户数量、物品数量、评分数量、评分范围和平均评分，并进一步计算用户-物品评分矩阵的密度。若将所有用户和所有物品构成一个完整评分矩阵，则理论上可能

存在 $|U| \times |I|$ 个评分位置，而训练集中实际只观察到 $|R|$ 条评分。因此评分矩阵密度定义为：

$$density = \frac{|R|}{|U| \times |I|} \quad (3)$$

对应的稀疏率为 $1 - density$ 。本数据集的基本规模如表 1 所示。

表 1: 训练集基本统计信息

统计项	数值
用户数	598
物品数	9077
评分数	90854
最低评分	10
最高评分	100
平均评分	69.8821
评分矩阵密度	1.6738%
评分矩阵稀疏率	98.3262%

统计结果显示，训练集中共有 90854 条评分，但相对于 598 个用户和 9077 个物品形成的完整评分空间，实际可见评分只占 1.6738%。该任务具有典型的稀疏推荐特征：绝大多数用户-物品组合没有真实评分，模型必须根据有限历史评分推断未知组合的分数。

这一特征对模型实现有直接影响。首先，不能把未出现的用户-物品组合当作 0 分处理，否则会把缺失值误认为负反馈，破坏评分预测的含义。其次，稀疏数据会使复杂模型更容易过拟合少量观测样本，因此模型需要正则化和稳定的回退策略。Baseline 模型先从全局平均分、用户偏置和物品偏置出发，提取最稳定的评分结构，为后续的协同过滤和矩阵分解方法提供基础。

1.3 用户与物品评分分布分析

除了整体规模外，用户和物品的评分数量分布也会影响推荐算法的稳定性。表 2 给出了训练集中用户侧和物品侧的评分数量统计。

表 2: 用户与物品评分数量分布

统计项	用户侧	物品侧
最少评分数量	15	1
最多评分数量	2678	280
平均评分数量	151.93	10.01
评分数量不超过 5 的对象数	0	5959
评分数量不超过 10 的对象数	0	7079

从用户侧看，每个训练用户至少有 15 条评分，平均每个用户约有 151.93 条评分，说明训练集中的用户画像相对较充分。也就是说，对于大多数训练用户，模型可以从其历史评分中估计出一定程度的打分习惯或兴趣倾向。

从物品侧看，数据呈现明显的长尾特征。9077 个训练物品中，有 5959 个物品的评分数量不超过 5，占训练物品数的 65.65%；有 7079 个物品的评分数量不超过 10，占训练物品数的 77.99%；其中只被评分 1 次的物品达到 3019 个，占 33.26%。大量物品只拥有很少的历史评分。对于这类长尾物品，如果直接根据少量评分估计物品质量或物品相似度，结果会很不稳定。

该分布特征说明，推荐模型不能只依赖物品之间的共现或相似关系。对于评分次数较少的物品，模型需要能够退回到更稳健的统计估计。Baseline 中的物品偏置正则化正是针对这一问题设计：当一个物品评分数量很少时，它的物品偏置会被压向 0，避免少量评分把该物品估计得过高或过低。

1.4 评分取值分布与预测边界

训练集中评分范围为 10 到 100，平均评分为 69.8821。进一步统计可得，评分不低于 80 的记录有 43444 条，占全部训练评分的 47.82%；评分不低于 90 的记录有 19267 条，占 21.21%。评分整体并不是围绕区间中点均匀分布，而是存在一定的高分倾向。

这种评分分布对模型有两点影响。第一，全局平均分本身已经是一个有意义的基准预测值，因为它反映了数据集中整体偏高的评分水平。第二，模型输出必须限制在合理评分区间内。若某些用户偏置或物品偏置叠加后导致预测值低于 10 或高于 100，就会产生不符合数据定义的结果。因此，最终预测阶段需要将模型输出截断到训练集观察到的评分范围内。

1.5 测试集冷启动情况与输出要求

在推荐系统中，冷启动通常指模型面对训练阶段没有足够历史行为的对象时，无法直接利用个性化信息进行预测的情况。本实验中的冷启动不是指程序需要执行某个额外操作，而是指 `Test.txt` 中部分待预测用户或物品没有出现在 `Train.txt` 的评分记录中。具体来说，如果某个测试用户从未在训练集中评分，则模型无法学习该用户的用户偏置，也无法获得该用户的历史评分列表；如果某个测试物品从未在训练集中被评分，则模型无法学习该物品的物品偏置，也无法为其建立 ItemCF 相似邻居。

`Test.txt` 中共有 610 个用户块和 9982 个待预测用户-物品对。与训练集对比后可以发现，测试集中存在 12 个训练集中未出现过的用户，占测试用户数的 1.97%；存在 647 个训练集中未出现过的物品，占测试物品数的 17.88%。从待预测评分对角度看，9028 个用户-物品对的用户和物品都在训练集中出现过，占 90.44%；另有 954 个预测对至少包含未见用户或未见物品，占 9.56%。

表 3: 测试集与训练集重合情况

统计项	数值
测试集用户块数	610
待预测用户-物品对数量	9982
训练集中未出现的测试用户数	12
训练集中未出现的测试物品数	647
用户和物品均已见的预测对数量	9028
至少包含未见用户或未见物品的预测对数量	954

这一统计结果说明，最终预测中不仅存在普通的已知用户、已知物品评分预测，也存在一定比例的冷启动或半冷启动情况。所谓半冷启动，是指用户和物品中只有一侧在训练集中出现过。例如，若用户已见但物品未见，模型仍然可以利用该用户的打分习惯，却无法利用该物品的历史质量信息；若用户未见但物品已见，则模型可以利用物品整体偏置，却无法利用用户个人偏好。若用户和物品均未见，则模型只能退回到全局平均评分这一最基础的信息。

这类样本会直接影响算法设计。Baseline 模型在遇到未见用户时将用户偏置视为 0，在遇到未见物品时将物品偏置视为 0，因此仍然可以基于全局平均分和另一侧已有偏置信息输出预测。ItemCF 对冷启动更加敏感，因为它需要查询目标用户的历史评分和目标物品的相似邻居；如果用户没有历史评

分，或者物品没有在训练阶段形成相似邻居，ItemCF 就无法计算相似物品残差修正。因此，本实验在 ItemCF 中保留 Baseline 作为回退模型，保证所有测试样本都能得到预测分数。

表 4: 不同测试样本类型的预测信息与回退策略

样本类型	可用信息	主要回退策略
用户和物品均已见	用户历史评分、物品历史评分、偏置项、相似邻居和隐向量均可用	正常调用各模型预测，融合模型对 ItemCF、矩阵分解和 UserCF 的输出加权平均。
用户已见、物品未见	可使用用户偏置和用户历史评分，但缺少物品偏置、物品相似邻居和物品隐向量	Baseline 使用全局平均分和用户偏置；ItemCF 找不到目标物品邻居时回退到 Baseline；矩阵分解不使用物品隐向量点积。
用户未见、物品已见	可使用物品偏置和物品历史评分，但缺少用户偏置、用户历史评分和用户隐向量	Baseline 使用全局平均分和物品偏置；ItemCF 缺少该用户历史评分时回退到 Baseline；矩阵分解不使用用户隐向量点积。
用户和物品均未见	只能使用训练集整体评分分布	各模型最终退回全局平均分，并将预测值限制在训练集评分范围内。

最终结果文件必须严格保持与 `Test.txt` 一致的用户块结构。也就是说，输出文件中每个用户块的用户编号、物品数量和物品顺序应与测试集对应位置一致，只是在每个物品编号后补充预测评分。程序生成的 `final_result.txt` 共包含 610 个用户块和 9982 条预测结果，用户块顺序、物品数量和物品顺序均与 `Test.txt` 保持一致，预测评分被限制在训练集评分范围 10 到 100 内。

2 Baseline 算法详细说明

2.1 算法思想

Baseline 模型是推荐系统评分预测中常用的基础模型。它不直接建模复杂的用户兴趣关系或物品相似关系，而是先解释评分数据中最稳定、最普遍的部分：全局平均评分水平、用户整体打分习惯以及物品整体受欢迎程度。

在实际评分数据中，不同用户的打分尺度往往不同。有的用户习惯给较高分，有的用户则习惯给较低分；同时，不同物品本身的平均受欢迎程度也不同，有的物品整体更容易获得高分，有的物品则整体评分偏低。因此，仅使用全局平均分预测所有评分是不够的。Baseline 模型在全局平均分的基础上加入用户偏置和物品偏置，使预测结果能够同时反映用户和物品两个方面的系统性差异。

本实验中 Baseline 模型的预测公式为：

$$\hat{r}_{ui} = \mu + b_u + b_i \quad (4)$$

其中， $\hat{r}_{ui}$ 表示用户 u 对物品 i 的预测评分， μ 表示训练集中所有评分的全局平均值， b_u 表示用户 u 的用户偏置， b_i 表示物品 i 的物品偏置。

2.2 全局平均分

全局平均分 μ 是 Baseline 模型中最基础的部分，计算方式为：

$$\mu = \frac{1}{|R|} \sum_{(u,i) \in R} r_{ui} \quad (5)$$

其中, R 表示训练集中所有已知评分记录的集合。全局平均分可以看作在完全不了解具体用户和物品时的基础预测值。在本实验数据中, 训练集平均评分为 69.8821, 因此如果不考虑用户和物品差异, 最简单的预测就是接近该平均值。

2.3 程序结构与核心参数

Baseline 模型在代码中实现为 `BaselineModel` 类。该类主要保存正则化参数、迭代轮数、全局平均分、用户偏置表和物品偏置表。核心参数设置如代码片段 1 所示。

Listing 1: Baseline 模型核心参数与状态变量

```

1 class BaselineModel:
2     def __init__(
3         self,
4         reg_user: float = 10.0,
5         reg_item: float = 15.0,
6         n_iters: int = 15,
7         seed: int = 2026,
8     ) -> None:
9         self.reg_user = reg_user
10        self.reg_item = reg_item
11        self.n_iters = n_iters
12        self.seed = seed
13
14        self.global_mean: float = 0.0
15        self.user_bias: Dict[int, float] = {}
16        self.item_bias: Dict[int, float] = {}
17        self.min_rating: float = 0.0
18        self.max_rating: float = 0.0

```

其中, `reg_user` 和 `reg_item` 分别对应用户偏置和物品偏置的正则化系数, `n_iters` 表示交替更新轮数。代码中将用户偏置和物品偏置保存为字典结构, 键为原始用户编号或物品编号, 值为对应偏置。这样做不要求用户编号或物品编号连续, 可以直接适配原始数据中较大的物品编号。

2.4 训练数据组织

为了高效更新用户偏置和物品偏置, 程序在训练开始时分别构建按用户分组和按物品分组的评分记录。按用户分组的数据用于更新 b_u , 按物品分组的数据用于更新 b_i 。相关代码如代码片段 2 所示。

Listing 2: 训练评分的用户分组与物品分组

```

1 user_ratings: Dict[int, List[Tuple[int, float]]] = defaultdict(list)
2 item_ratings: Dict[int, List[Tuple[int, float]]] = defaultdict(list)
3 ratings = []
4
5 for user_id, item_id, rating in train_data:
6     user_ratings[user_id].append((item_id, rating))
7     item_ratings[item_id].append((user_id, rating))
8     ratings.append(rating)
9

```



```

10 self.global_mean = sum(ratings) / len(ratings)
11 self.min_rating = min(ratings)
12 self.max_rating = max(ratings)
13 self.user_bias = {user_id: 0.0 for user_id in user_ratings}
14 self.item_bias = {item_id: 0.0 for item_id in item_ratings}

```

这段代码对应了 Baseline 训练的初始化阶段。程序首先遍历训练集中全部 (u, i, r_{ui}) 三元组，将同一个用户的评分记录放入 `user_ratings`，将同一个物品的评分记录放入 `item_ratings`。随后计算全局平均分 μ ，记录训练集评分上下界，并将所有用户偏置和物品偏置初始化为 0。这个初始化使第一轮用户偏置更新可以从全局平均分出发，之后再通过多轮交替更新逐步修正。

2.5 用户偏置

用户偏置 b_u 用来描述用户 u 整体上相对于平均水平更倾向于打高分还是低分。若某个用户经常给出高于平均水平的评分，则其用户偏置通常为正；若某个用户打分较严格，则其用户偏置可能为负。

在固定当前物品偏置 b_i 的情况下，用户偏置的更新公式为：

$$b_u = \frac{\sum_{i \in I(u)} (r_{ui} - \mu - b_i)}{\lambda_u + |I(u)|} \quad (6)$$

其中， $I(u)$ 表示用户 u 评分过的物品集合， $|I(u)|$ 表示该用户的评分数量， λ_u 是用户偏置的正则化系数。本实验代码中 λ_u 取值为 10.0。

该公式的含义是：对于用户 u 的所有历史评分，先扣除全局平均分和对应物品偏置，剩余部分可以理解为该用户自身打分习惯造成的偏离。对这些偏离进行求和并除以带正则化项的分母，就得到用户偏置。正则化项的作用是防止评分数量很少的用户产生过大的偏置，从而提高模型的稳定性。

2.6 物品偏置

物品偏置 b_i 用来描述物品 i 整体上相对于平均水平更容易获得高分还是低分。若一个物品在扣除用户打分习惯后仍然普遍获得较高评分，则其物品偏置通常为正；反之，则可能为负。

在固定当前用户偏置 b_u 的情况下，物品偏置的更新公式为：

$$b_i = \frac{\sum_{u \in U(i)} (r_{ui} - \mu - b_u)}{\lambda_i + |U(i)|} \quad (7)$$

其中， $U(i)$ 表示评价过物品 i 的用户集合， $|U(i)|$ 表示该物品被评分的次数， λ_i 是物品偏置的正则化系数。本实验代码中 λ_i 取值为 15.0。

该公式表示：对于物品 i 的全部评分记录，扣除全局平均分和对应用户偏置后，剩余部分反映了物品自身对评分的系统性影响。由于部分物品评分数量较少，如果直接根据少量评分估计物品质量，容易产生较大波动，因此分母中加入正则化项对物品偏置进行收缩。

2.7 交替迭代训练过程

用户偏置和物品偏置之间存在相互依赖关系。计算用户偏置时需要使用当前物品偏置，而计算物品偏置时又需要使用当前用户偏置，因此不能在一开始一次性直接得到二者的稳定取值。代码中采用交替迭代的方式训练 Baseline 模型。

训练过程首先计算全局平均分 μ ，并将所有用户偏置和物品偏置初始化为 0。随后重复进行多轮更新：每一轮先固定当前物品偏置，更新所有用户偏置；再固定更新后的用户偏置，更新所有物品偏置。本实验代码中迭代轮数固定为 15 轮。

迭代轮数属于训练过程中的超参数。若迭代轮数过少，用户偏置和物品偏置还没有充分互相校正，模型可能停留在较粗糙的偏置估计；若迭代轮数过多，虽然不会明显增加模型空间，但会带来额外训练时间，而且在收敛后继续迭代对 RMSE 的改善很小。因此，代码中的 15 轮并不是任意指定，而是在实验部分通过不同迭代轮数对比后选取的折中设置。

算法 1 Baseline 模型训练过程

输入：训练评分集合 R ，用户正则化系数 λ_u ，物品正则化系数 λ_i ，迭代轮数 T

输出：全局平均分 μ ，用户偏置 b_u ，物品偏置 b_i

```

1: 计算全局平均分  $\mu$ 
2: 初始化所有用户偏置  $b_u \leftarrow 0$ 
3: 初始化所有物品偏置  $b_i \leftarrow 0$ 
4: for  $t = 1$  to  $T$  do
5:   for 每一个用户  $u$  do
6:     根据当前物品偏置更新  $b_u$ 
7:   end for
8:   for 每一个物品  $i$  do
9:     根据当前用户偏置更新  $b_i$ 
10:  end for
11: end for
12: return  $\mu, b_u, b_i$ 

```

代码中的实际更新过程如代码片段 3 所示。每一轮迭代中，程序先遍历所有用户，根据当前物品偏置计算该用户的残差和，并更新用户偏置；随后遍历所有物品，根据最新用户偏置计算物品残差和，并更新物品偏置。

Listing 3: Baseline 偏置项交替更新代码

```

1  for _ in range(self.n_iters):
2      for user_id, records in user_ratings.items():
3          residual_sum = sum(
4              rating - self.global_mean - self.item_bias.get(item_id, 0.0)
5              for item_id, rating in records
6          )
7          self.user_bias[user_id] = residual_sum / (
8              self.reg_user + len(records)
9          )
10
11     for item_id, records in item_ratings.items():
12         residual_sum = sum(
13             rating - self.global_mean - self.user_bias.get(user_id, 0.0)
14             for user_id, rating in records
15         )
16         self.item_bias[item_id] = residual_sum / (
17             self.reg_item + len(records)
18         )

```

代码中的 `residual_sum` 与前文公式中的残差求和一致。更新用户偏置时，残差项为 $r_{ui} - \mu - b_i$ ；

更新物品偏置时，残差项为 $r_{ui} - \mu - b_u$ 。分母中的 $\text{len}(\text{records})$ 分别表示当前用户的评分数量或当前物品的被评分数量， reg_user 和 reg_item 则对应正则化项。由此可见，代码实现与公式推导是一一对应的。

从优化角度看，该模型等价于在正则化约束下最小化真实评分与预测评分之间的平方误差：

$$\min_{b_u, b_i} \sum_{(u,i) \in R} (r_{ui} - \mu - b_u - b_i)^2 + \lambda_u \sum_u b_u^2 + \lambda_i \sum_i b_i^2 \quad (8)$$

交替更新的每一步都在固定另一类偏置的条件下求当前偏置的较优解。随着迭代进行，用户偏置和物品偏置会互相校正并逐渐稳定。正则化项会将偏置向 0 收缩，避免由于评分样本过少导致参数过大，因此也有助于训练过程保持稳定。

2.8 预测与边界处理

训练完成后，对于任意用户-物品对 (u, i) ，Baseline 模型使用下式进行评分预测：

$$\hat{r}_{ui} = \mu + b_u + b_i \quad (9)$$

如果待预测用户没有在训练集中出现，则代码无法得到该用户的历史打分偏置，此时将 b_u 视为 0；如果待预测物品没有在训练集中出现，则将 b_i 视为 0。这种处理方式使 Baseline 模型具备较稳定的回退能力，即使面对未见用户或未见物品，也可以给出基于全局平均分和已有偏置信息的预测结果。

预测函数的核心代码如代码片段 4 所示。

Listing 4: Baseline 预测与冷启动回退

```
1 def predict(self, user_id: int, item_id: int) -> float:
2     return (
3         self.global_mean
4         + self.user_bias.get(user_id, 0.0)
5         + self.item_bias.get(item_id, 0.0)
6     )
```

这里使用字典的 `get` 方法读取偏置项。当用户或物品没有在训练集中出现时，`get` 会返回默认值 0.0，因此模型会自动退回到全局平均分和另一侧已有偏置。这一点与前文测试集冷启动统计相对应：由于测试集中存在未见用户和未见物品，预测函数必须具备这种回退能力，否则部分测试样本将无法正输出评分。

在最终输出预测结果时，代码还会将预测评分限制在训练集评分范围内，即不低于 10 且不高于 100。这样可以保证输出结果符合数据集中评分取值范围，避免模型给出不合理的越界分数。

3 ItemCF 算法详细说明

3.1 算法思想

ItemCF 即基于物品的协同过滤算法。它的核心思想是：如果用户已经对若干物品表现出较高偏好，那么该用户也可能对与这些物品相似的其他物品给出较高评分。与基于用户的协同过滤相比，ItemCF 更关注物品之间的稳定关系。用户兴趣可能随时间变化，而由共同评分行为反映出的物品相似关系通常更稳定，因此该方法适合用于评分预测任务。

对于目标用户 u 和目标物品 i ，ItemCF 会从用户 u 已经评分过的物品中寻找与物品 i 相似的邻居物品，并利用这些邻居物品上的评分信息推断 u 对 i 的评分。朴素形式可以写为：

$$\hat{r}_{ui} = \frac{\sum_{j \in N_i(u)} \text{sim}(i, j) r_{uj}}{\sum_{j \in N_i(u)} |\text{sim}(i, j)|} \quad (10)$$

其中， $N_i(u)$ 表示用户 u 已评分且与目标物品 i 相似的邻居物品集合， $\text{sim}(i, j)$ 表示物品 i 与物品 j 的相似度。该公式说明，相似度越高的历史物品，对目标物品预测结果的影响越大。

在本实验中，ItemCF 并不直接使用原始评分计算相似度。原始评分会受到用户打分尺度和物品整体热度影响，例如某些用户习惯给高分，某些物品本身整体评分较高。如果直接比较原始评分，模型可能把这些系统性因素误认为物品之间的真实相似。因此，程序在 Baseline 预测基础上构造残差，并在残差空间中计算物品相似度。

3.2 程序结构

item_cf.py 中的 ItemCF 实现由 ItemCFModel 类完成。主要结构如表 5 所示。

表 5: ItemCF 实现中的主要结构

结构或变量	作用
baseline	保存 Baseline 模型，用于计算基础预测分和冷启动回退。
user_ratings	保存每个用户的历史评分，预测时从中查找用户已评分物品。
item_similarities	保存每个物品的相似邻居及相似度。
residual_matrix	保存用户-物品残差矩阵，用于计算物品相似度。
mask_matrix	标记真实评分是否存在，用于统计共同评分用户数。

模型核心参数包括 top_k、similarity_top_n、shrinkage、min_common 和 block_size。其中 top_k 控制预测阶段使用的相似邻居数量，similarity_top_n 控制每个物品保留的相似邻居数量，shrinkage 和 min_common 用于控制相似度可靠性，block_size 控制分块计算相似度时的临时矩阵规模。这些参数的取值会在准确率、训练时间和模型占用之间形成权衡，实验部分将结合验证结果进行比较。

3.3 残差矩阵构建

ItemCF 训练的第一步是得到 Baseline 基础预测，并计算每条真实评分的残差：

$$e_{ui} = r_{ui} - \text{baseline}(u, i) \quad (11)$$

其中， e_{ui} 表示用户 u 对物品 i 的真实评分相对于基础预测的偏离。若 $e_{ui} > 0$ ，说明用户对该物品的评价高于基础模型预期；若 $e_{ui} < 0$ ，说明用户对该物品的评价低于基础模型预期。代码中首先训练 Baseline，并建立用户编号和物品编号到矩阵下标的映射：

Listing 5: 训练 Baseline 并建立矩阵下标映射

```

1 self.baseline.fit(train_data)
2 user_ids = sorted({user_id for user_id, _, _ in train_data})
3 item_ids = sorted({item_id for _, item_id, _ in train_data})
4 user_to_idx = {user_id: idx for idx, user_id in enumerate(user_ids)}
5 item_to_idx = {item_id: idx for idx, item_id in enumerate(item_ids)}
```

该片段先复用前一节的 Baseline 模型得到基础预测器，再将原始用户编号和物品编号压缩成连续下标。由于物品编号最大值远大于物品实际数量，使用连续下标可以避免直接按原始编号建立过大的矩阵。

随后程序构造残差矩阵和评分存在标记矩阵：

Listing 6: 残差矩阵与评分标记矩阵初始化

```
1 residual_matrix = np.zeros((n_users, n_items), dtype=np.float32)
2 mask_matrix = np.zeros((n_users, n_items), dtype=np.float32)
3 self.user_ratings = {user_id: {} for user_id in user_ids}
```

`residual_matrix` 保存残差 e_{ui} ，`mask_matrix` 只记录某个用户-物品位置是否存在真实评分。后者不保存评分值，而是用于统计两个物品的共同评分用户数。

对每条训练评分，程序计算 Baseline 残差并写入矩阵：

Listing 7: 写入用户-物品残差

```
1 baseline_score = self.baseline.predict(user_id, item_id)
2 residual_matrix[user_idx, item_idx] = rating - baseline_score
3 mask_matrix[user_idx, item_idx] = 1.0
4 self.user_ratings[user_id][item_id] = rating
```

这几行代码对应公式 $e_{ui} = r_{ui} - \text{baseline}(u, i)$ 。同时，用户历史评分被保存到 `user_ratings` 中，供预测阶段从用户已评分物品中寻找相似邻居。

3.4 物品相似度计算

程序将每个物品表示为一个残差向量。向量维度对应用户，向量元素为用户在该物品上的残差。物品 i 和物品 j 的相似度采用余弦相似度：

$$\text{sim}(i, j) = \frac{\sum_{u \in U(i) \cap U(j)} e_{ui} e_{uj}}{\sqrt{\sum_u e_{ui}^2} \sqrt{\sum_u e_{uj}^2}} \quad (12)$$

其中， $U(i) \cap U(j)$ 表示同时评价过两个物品的用户集合。若两个物品的残差向量方向相近，说明用户对它们的个性化偏离模式相似。

在实现中，代码先计算每个物品残差向量的范数：

Listing 8: 计算物品残差向量范数

```
1 norms = np.sqrt(np.sum(residual_matrix * residual_matrix, axis=0))
```

该变量对应余弦相似度分母中的向量长度。随后，程序按物品块计算相似度分子和共同评分用户数：

Listing 9: 分块计算点积与共同评分数

```
1 block_residuals = residual_matrix[:, start:end]
2 block_masks = mask_matrix[:, start:end]
3 numerators = block_residuals.T @ residual_matrix
4 common_counts = block_masks.T @ mask_matrix
```

`numerators` 对应物品残差向量之间的点积, `common_counts` 对应两个物品被同一批用户共同评分的数量。按块计算可以避免一次性构造完整相似度矩阵, 从而降低临时内存压力。

对于共同评分用户数不足或残差向量范数为 0 的物品对, 程序不计算有效相似度:

Listing 10: 相似度有效性判断

```
1 denom = norms[item_idx] * norms
2 valid = (denom > 1e-12) & (
3     common_counts[row_offset] >= self.min_common
4 )
```

这里的 `min_common` 用于排除共同评分用户过少的物品对, `denom > 1e-12` 用于避免除以接近 0 的范数。通过这两个条件, 模型只保留具有基本统计支撑的相似度计算。

在计算余弦相似度后, 程序进一步使用 `shrinkage` 对相似度进行收缩:

Listing 11: 余弦相似度与 shrinkage 修正

```
1 sims[valid] = numerators[row_offset, valid] / denom[valid]
2 sims[valid] *= common_counts[row_offset, valid] / (
3     common_counts[row_offset, valid] + self.shrinkage
4 )
```

对应的数学形式为:

$$sim'(i, j) = sim(i, j) \times \frac{c_{ij}}{c_{ij} + \gamma} \quad (13)$$

其中, c_{ij} 为共同评分用户数, γ 为收缩系数。共同评分用户数越少, 修正因子越小, 相似度被压得越低; 共同评分用户数越多, 相似度越接近原始余弦相似度。具体参数取值将在实验部分结合 RMSE 和运行开销进一步比较。

3.5 相似邻居保留

计算得到当前物品与其他物品的相似度后, 程序并不保存完整相似度向量, 而是只保留绝对值较大的邻居:

Listing 12: 截取相似度较强的邻居

```
1 if nonzero_indices.size > self.similarity_top_n:
2     abs_values = np.abs(sims[nonzero_indices])
3     local_idx = np.argpartition(abs_values, -self.similarity_top_n)[
4         -self.similarity_top_n :
5     ]
6     nonzero_indices = nonzero_indices[local_idx]
```

这段代码通过 `argpartition` 找到相似度绝对值较大的候选邻居, 避免对完整数组做全量排序。由于训练集中物品数较多, 若保存所有物品对相似度, 会带来较高存储开销; 只保存强邻居既能减少存储量, 也能降低弱相关物品对预测的干扰。

最终保存时, 程序按相似度绝对值降序组织邻居:

Listing 13: 按相似度强度保存邻居


```

1 ordered = sorted(
2     nonzero_indices.tolist(),
3     key=lambda idx: abs(float(sims[idx])),
4     reverse=True,
5 )

```

这样在预测阶段可以优先使用最强的相似物品，符合 ItemCF 中“强相似邻居贡献更大”的思想。

3.6 基于相似邻居的评分预测

预测阶段首先得到目标用户-物品对的 Baseline 基础分，并读取用户历史评分和目标物品的相似邻居：

Listing 14: 读取基础分与候选邻居

```

1 baseline_score = self.baseline.predict(user_id, item_id)
2 user_history = self.user_ratings.get(user_id)
3 item_neighbors = self.item_similarities.get(item_id)

```

如果用户没有历史评分，或者目标物品没有相似邻居，模型直接返回 Baseline 预测。这是 ItemCF 面对冷启动或邻居缺失时的回退策略。这里的“没有历史评分”对应测试用户在训练集中没有评分记录，程序无法从 `user_ratings` 中找到该用户；“没有相似邻居”则对应目标物品没有出现在训练集，或者虽然出现过但没有满足共同评分阈值的相似物品。

对于用户历史评分中与目标物品存在相似关系的物品，程序计算该邻居物品上的残差：

Listing 15: 从用户历史中提取可用邻居残差

```

1 similarity = item_neighbors.get(neighbor_item)
2 neighbor_baseline = self.baseline.predict(user_id, neighbor_item)
3 residual = rating - neighbor_baseline
4 weighted_neighbors.append((abs(similarity), similarity, residual))

```

这段代码说明，预测阶段并不是直接使用用户对邻居物品的原始评分，而是继续使用相对于 Baseline 的残差。这样可以保持训练阶段和预测阶段的一致性。

最终，程序按相似度绝对值排序，取前 `top_k` 个邻居进行加权残差修正：

Listing 16: top-k 邻居加权残差预测

```

1 weighted_neighbors.sort(key=lambda x: x[0], reverse=True)
2 top_neighbors = weighted_neighbors[: self.top_k]
3 numerator = sum(sim * residual for _, sim, residual in top_neighbors)
4 denominator = sum(weight for weight, _, _ in top_neighbors)

```

对应的预测公式为：

$$\hat{r}_{ui} = baseline(u, i) + \frac{\sum_{j \in K_i(u)} sim(i, j) e_{uj}}{\sum_{j \in K_i(u)} |sim(i, j)|} \quad (14)$$

其中， $K_i(u)$ 表示用户 u 的历史评分中与目标物品 i 最相关的前 k 个邻居物品。该公式说明，ItemCF 先使用 Baseline 给出稳定基础分，再根据用户在相似物品上的个性化残差进行修正。

3.7 回退机制与边界处理

ItemCF 依赖用户历史评分和物品邻居信息。当用户没有历史评分、目标物品没有相似邻居、用户历史物品与目标物品之间没有可用相似关系，或者加权分母过小时，模型无法可靠地计算残差修正。此时程序直接返回 Baseline 基础分：

$$\hat{r}_{ui} = baseline(u, i) \quad (15)$$

这种回退机制与本数据集的冷启动特征相匹配。前文统计显示，测试集中存在一定数量的未见用户、未见物品以及至少包含一侧未见对象的预测对。对于这些样本，ItemCF 可能无法找到有效相似邻居，而 Baseline 可以通过全局平均分和已有偏置给出稳定预测。

更具体地说，当用户未见而物品已见时，Baseline 仍可使用物品偏置和全局平均分；当用户已见而物品未见时，Baseline 仍可使用用户偏置和全局平均分；当用户与物品均未见时，模型至少可以退回到全局平均分。ItemCF 只有在用户存在历史评分且目标物品存在可用相似邻居时，才在 Baseline 基础上进行残差修正。因此，本实验中的 ItemCF 实际上是一个以 Baseline 为基础、在有可靠邻居信息时进行个性化修正的混合模型。

最终输出时，ItemCF 的预测结果同样会被限制在训练集评分范围内，即 10 到 100 之间。这样既保持了结果格式的合法性，也避免相似度加权修正造成越界评分。

4 Matrix Factorization 算法详细说明

4.1 算法思想

Matrix Factorization 即矩阵分解算法，是推荐系统评分预测中的经典方法。它的基本思想是：用户对物品的偏好可以由若干个潜在因素共同决定。这里的“潜在因素”并不是数据集中已经给出的类别字段，也不是人工事先规定的标签，而是模型为了降低预测误差而自动学习出来的数值表示。

在矩阵分解中，第一次需要引入两个核心概念：用户向量和物品向量。用户向量是模型为每个用户学习到的一组连续数值，记为 p_u ，用于表示用户 u 在隐空间中的兴趣位置；物品向量是模型为每个物品学习到的一组连续数值，记为 q_i ，用于表示物品 i 在同一个隐空间中的特征位置。二者维度相同，例如都可以是 5 维、20 维或更高维，这样才能通过点积计算用户和物品之间的匹配程度。

用户向量不是由用户的姓名、编号或人工标签直接决定的，物品向量也不是由物品名称或类别直接决定的。在本实验的数据中，模型能够利用的主要信息是用户-物品评分记录。因此，用户向量和物品向量一开始只是随机初始化的数值，随后在训练过程中根据评分误差不断调整。某个用户给哪些物品高分、给哪些物品低分，会共同决定该用户向量最终移动到隐空间中的什么位置；某个物品被哪些用户喜欢、被哪些用户不喜欢，也会共同决定该物品向量最终形成什么表示。

具体来说，矩阵分解为每个用户学习一个用户隐向量 p_u ，为每个物品学习一个物品隐向量 q_i 。需要注意的是，隐向量的每一维通常不具有固定的人类可解释含义。例如，不能简单地认为第 1 维一定表示“动作偏好”、第 2 维一定表示“文艺偏好”、第 3 维一定表示“受欢迎程度”。模型只保证这些维度组合起来能够帮助预测评分，并不保证单独某一维可以被直接命名。

更准确地说，隐向量整体构成了一个低维兴趣空间。在这个空间中，如果某个用户向量和某个物品向量方向接近、点积较大，模型就会认为用户与物品更匹配；如果二者方向不一致或点积较小，模型就会认为匹配程度较低。经过大量评分样本训练后，具有相似评分模式的用户会逐渐形成相近的用户向量，常被相同用户群体喜欢的物品也会逐渐形成相近的物品向量。因此，隐向量中的“语义”不是人工指定进去的，而是由评分数据中的共现关系和误差更新过程自动形成的潜在偏好结构。

可以用二维隐向量的简化例子说明这一训练机制。设用户 A 和物品 X 的隐向量分别为：

$$p_A = \begin{bmatrix} 0.2 & 0.8 \end{bmatrix}, \quad q_X = \begin{bmatrix} 0.3 & 0.4 \end{bmatrix} \quad (16)$$

暂时忽略用户偏置和物品偏置，只假设基础分为 70，则当前预测为：

$$\hat{r}_{AX} = 70 + p_A^T q_X = 70 + (0.2 \times 0.3 + 0.8 \times 0.4) = 70.38 \quad (17)$$

如果真实评分为 90，则误差为：

$$error = r_{AX} - \hat{r}_{AX} = 90 - 70.38 = 19.62 \quad (18)$$

该误差为正，说明模型低估了用户 A 对物品 X 的喜欢程度。在不考虑正则化时，SGD 会按照物品向量方向调整用户向量，同时按照用户向量方向调整物品向量：

$$p_A \leftarrow p_A + \eta \cdot error \cdot q_X, \quad q_X \leftarrow q_X + \eta \cdot error \cdot p_A \quad (19)$$

若学习率 $\eta = 0.01$ ，则更新后的向量大致变为：

$$p_A = \begin{bmatrix} 0.25886 & 0.87848 \end{bmatrix}, \quad q_X = \begin{bmatrix} 0.33924 & 0.55696 \end{bmatrix} \quad (20)$$

更新后，二者点积会增大，下一次模型预测用户 A 对物品 X 的评分也会提高。这可以理解为：高于预测值的评分样本会推动用户向量和物品向量在隐空间中更加匹配。相反，如果真实评分低于预测值，则 **error** 为负，更新方向会降低用户向量与物品向量的点积，使二者匹配程度下降。经过大量高分和低分样本的反复作用，模型逐渐把评分模式相似的用户或物品放到隐空间中相近的位置，把评分模式差异较大的对象放到不同方向上。

因此，矩阵分解与神经网络中的 embedding 思路有相似之处：它同样为离散的用户编号和物品编号学习连续向量表示，并通过预测误差反复更新这些向量。但本实验中的矩阵分解模型没有多层非线性结构，也不直接输入物品文本、类别或图像信息；它只从用户-物品评分关系中学习隐空间结构。

最基础的矩阵分解预测可以写为 $p_u^T q_i$ ，即只用用户向量和物品向量的点积表示二者的匹配程度。但是在真实评分数据中，评分并不只由个性化匹配决定，还包含一些更加稳定的整体因素。首先，数据集本身存在一个整体评分水平，例如本实验训练集平均评分约为 69.8821，如果完全不考虑这一点，模型就需要让隐向量点积同时承担“整体平均分”和“个性化偏好”两种作用。其次，不同用户的打分尺度不同，有的用户习惯给高分，有的用户习惯给低分；不同物品的整体受欢迎程度也不同，有的物品普遍评分偏高，有的物品普遍评分偏低。

因此，引入偏置项的目的，是先把这些稳定、全局性的评分差异单独建模出来，让隐向量点积专注于刻画更细粒度的用户-物品匹配关系。全局平均分 μ 表示数据集中最基础的评分水平，用户偏置 b_u 表示用户 u 相对于平均水平的整体打分倾向，物品偏置 b_i 表示物品 i 相对于平均水平的整体受欢迎程度。这样，模型不需要用隐向量去重复学习“某个用户总是打高分”或“某个物品整体受欢迎”这类简单规律，而可以把隐向量容量更多用于表示个性化兴趣结构。

因此，本实验实现的是带偏置项的矩阵分解模型：

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i \quad (21)$$

其中， $\mu + b_u + b_i$ 与前文 Baseline 模型的含义一致，用来提供稳定的基础预测； p_u 和 q_i 分别表

示用户和物品的隐向量, $p_u^T q_i$ 则表示在扣除全局平均分、用户打分习惯和物品整体热度之后, 用户兴趣与物品特征之间剩余的个性化匹配程度。与 Baseline 相比, 矩阵分解多出的点积项可以进一步刻画“某个用户为什么会特别喜欢某些物品、而不是只按照用户平均打分和物品平均受欢迎程度预测”的细粒度关系。

4.2 程序结构

`matrix_factorization.py` 中使用 `MatrixFactorizationModel` 类实现带偏置矩阵分解。主要结构如表 6 所示。

表 6: Matrix Factorization 实现中的主要结构

结构或变量	作用
<code>user_to_idx</code>	将原始用户编号映射为连续数组下标。
<code>item_to_idx</code>	将原始物品编号映射为连续数组下标。
<code>user_bias</code>	保存每个用户的偏置项 b_u 。
<code>item_bias</code>	保存每个物品的偏置项 b_i 。
<code>user_factors</code>	保存用户隐向量矩阵 P 。
<code>item_factors</code>	保存物品隐向量矩阵 Q 。

模型核心参数包括 `n_factors`、`n_epochs`、`learning_rate`、`reg`、`reg_bias`、`init_std`、`init_with_baseline` 和 `freeze_bias`。其中 `n_factors` 控制隐向量维度, `n_epochs` 控制训练轮数, `learning_rate` 控制每次参数更新步长, `reg` 和 `reg_bias` 分别控制隐向量和偏置项的正则化强度。 `init_std` 控制随机隐向量的初始幅度, `init_with_baseline` 表示是否先用 Baseline 模型初始化 μ 、 b_u 和 b_i , `freeze_bias` 表示在 SGD 中是否固定这些偏置项。隐向量维度、学习率和正则化强度都是预设超参数, 会直接影响模型表达能力、训练时间和核心结构占用, 实验部分将进一步比较不同参数对模型表现的影响。

4.3 编号映射与数据准备

由于原始用户编号和物品编号不一定适合直接作为数组下标, 程序在训练开始时先建立连续编号映射:

Listing 17: 用户与物品编号映射

```

1 user_ids = sorted({user_id for user_id, _, _ in train_data})
2 item_ids = sorted({item_id for _, item_id, _ in train_data})
3 self.user_to_idx = {user_id: idx for idx, user_id in enumerate(user_ids)}
4 self.item_to_idx = {item_id: idx for idx, item_id in enumerate(item_ids)}
```

该做法与前文 ItemCF 中的矩阵下标压缩类似。连续下标可以使偏置向量和隐向量矩阵以数组方式访问, 提高访问效率, 同时避免按原始物品编号直接开辟过大数组。

随后, 程序提取评分数组并计算全局平均分和评分上下界:

Listing 18: 评分数组与全局统计量

```

1 ratings = np.array([rating for _, _, rating in train_data], dtype=np.float32)
2 self.global_mean = float(ratings.mean())
3 self.min_rating = float(ratings.min())
```

```
4 self.max_rating = float(ratings.max())
```

其中, `global_mean` 对应预测公式中的 μ , 评分上下界用于最终预测结果的边界截断。为了便于 SGD 逐条样本访问, 程序还将每条训练样本中的用户和物品转换为连续下标:

Listing 19: 训练样本下标数组

```
1 user_indices = np.array(
2     [self.user_to_idx[user_id] for user_id, _, _ in train_data],
3     dtype=np.int32,
4 )
5 item_indices = np.array(
6     [self.item_to_idx[item_id] for _, item_id, _ in train_data],
7     dtype=np.int32,
8 )
```

这样在训练循环中, 程序可以直接使用整数下标访问偏置项和隐向量, 而不需要反复查询原始编号。

4.4 参数初始化

矩阵分解需要学习用户偏置、物品偏置、用户隐向量和物品隐向量。如果四类参数全部从随机状态开始训练, 隐向量既要学习整体评分水平, 又要学习个性化交互关系, 容易使训练不稳定。结合 Baseline predictor 的思想, 代码中先训练 `BaselineModel`, 得到稳定的 μ 、 b_u 和 b_i , 再让矩阵分解中的隐向量主要学习 Baseline 不能解释的剩余部分。

Listing 20: 使用 Baseline 初始化矩阵分解偏置项

```
1 if self.init_with_baseline:
2     baseline = BaselineModel(seed=self.seed)
3     baseline.fit(train_data)
4     self.global_mean = baseline.global_mean
5     self.user_bias = np.array(
6         [baseline.user_bias.get(user_id, 0.0) for user_id in user_ids],
7         dtype=np.float32,
8     )
9     self.item_bias = np.array(
10        [baseline.item_bias.get(item_id, 0.0) for item_id in item_ids],
11        dtype=np.float32,
12    )
```

这段初始化的作用是把能够稳定解释的数据规律先交给 Baseline 部分处理: 全局平均分表示整体评分水平, 用户偏置表示用户打分尺度, 物品偏置表示物品整体受欢迎程度。这样, 隐向量点积 $p_u^T q_i$ 不需要重复学习这些基础因素, 而是更集中地刻画用户与物品之间的剩余交互关系。

隐向量仍然需要随机初始化。代码使用均值为 0、标准差由 `init_std` 控制的正态分布生成用户向量和物品向量:

Listing 21: 用户隐向量与物品隐向量初始化

```
1 self.user_factors = rng.normal(
2     loc=0.0,
```

```

3     scale=self.init_std,
4     size=(n_users, self.n_factors),
5 ).astype(np.float32)
6 self.item_factors = rng.normal(
7     loc=0.0,
8     scale=self.init_std,
9     size=(n_items, self.n_factors),
10 ).astype(np.float32)

```

隐向量不能全部初始化为 0，否则所有用户或物品在训练初期完全相同，SGD 更新也会缺少区分性。随机初始化可以打破这种对称性，使不同用户和物品在训练过程中逐渐学习到不同的隐空间表示。最终实验中 `init_std` 取 0.01，即采用较小幅度的随机初始化，避免随机点积在训练初期覆盖 Baseline 已经得到的稳定预测。代码固定随机种子为 2026，以保证实验结果可复现。

4.5 训练目标与 SGD 更新

矩阵分解的训练目标是使预测评分尽量接近真实评分，同时通过正则化限制参数规模。其目标函数可以写为：

$$\min \sum_{(u,i) \in R} (r_{ui} - \mu - b_u - b_i - p_u^T q_i)^2 + \lambda \sum_u \|p_u\|^2 + \lambda \sum_i \|q_i\|^2 + \lambda_b \sum_u b_u^2 + \lambda_b \sum_i b_i^2 \quad (22)$$

其中， λ 表示隐向量正则化系数， λ_b 表示偏置项正则化系数。正则化可以避免用户向量、物品向量或偏置项过大，降低数据稀疏场景下过拟合的风险。

训练时，程序使用随机梯度下降。每一轮训练前，先打乱训练样本顺序：

Listing 22: 每轮训练前打乱样本顺序

```

1 order = np.arange(len(train_data), dtype=np.int32)
2 learning_rate = self.learning_rate
3
4 for _ in range(self.n_epochs):
5     rng.shuffle(order)

```

随机打乱样本可以减少固定样本顺序对参数更新方向的影响，使 SGD 更新更稳定。对于每条训练样本，程序先计算预测值：

Listing 23: 矩阵分解单样本预测

```

1 pred = (
2     self.global_mean
3     + self.user_bias[user_idx]
4     + self.item_bias[item_idx]
5     + np.dot(self.user_factors[user_idx], self.item_factors[item_idx])
6 )
7 error = rating - pred

```

该片段直接对应公式 $\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$ 。其中 `error` 表示真实评分与预测评分之间的差异。若预测值偏低，误差为正；若预测值偏高，误差为负。

从隐空间角度看，`error` 的符号决定了用户向量和物品向量的调整方向。当 `error` 为正时，说明当前用户对当前物品的真实评分高于模型预期，模型需要提高二者的匹配程度；当 `error` 为负时，说明模型预测过高，需要降低二者的匹配程度。下面的偏置项更新和隐向量更新都围绕这一误差信号展开。

计算误差后，程序支持继续更新偏置项：

Listing 24: 偏置项 SGD 更新

```

1 if not self.freeze_bias:
2     self.user_bias[user_idx] += learning_rate * (
3         error - self.reg_bias * self.user_bias[user_idx]
4     )
5     self.item_bias[item_idx] += learning_rate * (
6         error - self.reg_bias * self.item_bias[item_idx]
7     )

```

其中，`error` 项推动偏置向减小预测误差的方向变化，`reg_bias` 项则将偏置向 0 收缩，防止偏置在少量样本上变得过大。不过，最终调优后的配置采用 `freeze_bias=True`，即固定由 Baseline 训练得到的 μ 、 b_u 和 b_i ，只更新用户隐向量与物品隐向量。这样设置的原因是本数据集评分矩阵较稀疏，Baseline 偏置已经较稳定；若在 SGD 中同时更新偏置和隐向量，二者可能重复解释同一部分误差，反而造成验证集效果下降。固定偏置后，矩阵分解更接近“Baseline 先给基础预测，隐向量只学习残差交互”的结构。

随后，程序更新用户隐向量和物品隐向量：

Listing 25: 隐向量 SGD 更新

```

1 user_vector = self.user_factors[user_idx].copy()
2 item_vector = self.item_factors[item_idx].copy()
3 self.user_factors[user_idx] += learning_rate * (
4     error * item_vector - self.reg * user_vector
5 )
6 self.item_factors[item_idx] += learning_rate * (
7     error * user_vector - self.reg * item_vector
8 )

```

这里先复制更新前的用户向量和物品向量，是为了保证两侧向量更新都基于同一时刻的参数。若直接在更新用户向量后再使用更新后的用户向量更新物品向量，会使一次样本更新中的梯度不再对应同一组旧参数。代码中的 `reg` 对应隐向量正则化项，用于限制向量长度。

该更新过程正对应前文对隐向量的解释。若某个用户给某个物品的评分高于当前预测值，`error * item_vector` 和 `error * user_vector` 会推动用户向量与物品向量在能够增大点积的方向上调整；若评分低于预测值，`error` 为负，更新方向则会降低二者点积。正则化项 `- self.reg * user_vector` 和 `- self.reg * item_vector` 会抑制向量无限增大，使模型不能只靠放大向量长度来降低训练误差。

每一轮训练结束后，程序衰减学习率：

Listing 26: 学习率衰减

```

1 learning_rate *= 0.95

```

学习率衰减使训练初期能够用较大步长快速调整参数，训练后期则用较小步长细化参数，减少震荡。

4.6 预测与冷启动处理

训练完成后，矩阵分解预测函数先以全局平均分作为基础分，再根据用户和物品是否已在训练集中出现，逐步加入偏置项和隐向量点积：

Listing 27: 矩阵分解预测入口

```
1 score = self.global_mean
2 user_idx = self.user_to_idx.get(user_id)
3 item_idx = self.item_to_idx.get(item_id)
```

如果用户或物品没有出现在训练集中，对应下标为 `None`，后续就不会使用该侧参数。对于已见用户和已见物品，程序分别加入偏置项：

Listing 28: 预测阶段加入已知偏置

```
1 if user_idx is not None and self.user_bias is not None:
2     score += float(self.user_bias[user_idx])
3 if item_idx is not None and self.item_bias is not None:
4     score += float(self.item_bias[item_idx])
```

只有当用户和物品都已见，并且隐向量矩阵已经训练完成时，模型才计算点积项：

Listing 29: 预测阶段加入隐向量点积

```
1 if user_idx is not None and item_idx is not None:
2     score += float(np.dot(
3         self.user_factors[user_idx],
4         self.item_factors[item_idx],
5     ))
```

这种设计使矩阵分解能够处理不同程度的冷启动情况。用户未见而物品已见时，模型可使用全局平均分和物品偏置；用户已见而物品未见时，模型可使用全局平均分和用户偏置；用户和物品都未见时，模型至少可以退回全局平均分。只有当用户和物品都在训练集中出现过时，隐向量点积才有意义。

4.7 核心结构占用估计

矩阵分解模型的主要持久化结构来自用户隐向量矩阵、物品隐向量矩阵、用户偏置、物品偏置以及编号映射。代码中通过 `approximate_size_bytes` 估算这些核心结构占用的空间：

Listing 30: 矩阵分解主要数组空间统计

```
1 if self.user_bias is not None:
2     size += self.user_bias.nbytes
3 if self.item_bias is not None:
4     size += self.item_bias.nbytes
5 if self.user_factors is not None:
6     size += self.user_factors.nbytes
7 if self.item_factors is not None:
8     size += self.item_factors.nbytes
```

其中，隐向量维度 `n_factors` 会直接影响 `user_factors` 和 `item_factors` 的大小。当维度增大时，模型表达能力增强，但训练时间和核心结构占用也会随之增加。因此，隐向量维度不仅是模型准确率参数，也是性能参数，实验部分将对其进行单独分析。

5 UserCF 算法详细说明

5.1 算法思想

UserCF 即基于用户的协同过滤算法。它的出发点是：如果两个用户在已经评分的物品上表现出相似的偏好，那么其中一个用户对某个物品的评分残差，可以为另一个用户预测该物品提供参考。与 ItemCF 关注“物品之间是否相似”不同，UserCF 关注“用户之间是否相似”。对于目标用户 u 和目标物品 i ，UserCF 会寻找与 u 相似、且已经评价过物品 i 的用户集合，并利用这些相似用户在物品 i 上的评分信息修正基础预测。

在本实验中，UserCF 同样不直接在原始评分上计算用户相似度，而是在 Baseline 残差空间中工作。这样做的原因与 ItemCF 相同：不同用户存在打分尺度差异，不同物品也存在整体受欢迎程度差异。先用 Baseline 解释全局平均分、用户偏置和物品偏置后，剩余残差更能反映用户之间真正相近或相反的个性化偏好。用户 u 对物品 i 的预测可以写为：

$$\hat{r}_{ui}^{UserCF} = b_{ui} + \frac{\sum_{v \in N_u(i)} sim(u, v)(r_{vi} - b_{vi})}{\sum_{v \in N_u(i)} |sim(u, v)|} \quad (23)$$

其中， b_{ui} 表示 Baseline 对用户 u 和物品 i 的基础预测， $N_u(i)$ 表示与用户 u 相似且评价过物品 i 的邻居用户集合， $sim(u, v)$ 表示用户 u 和用户 v 在残差向量上的相似度。分子部分累加相似用户对目标物品的残差评分，分母使用相似度绝对值进行归一化。若目标用户没有相似用户，或者目标物品没有被相似用户评分，模型会回退到 Baseline 预测。

5.2 程序实现

UserCF 在 `user_cf.py` 中实现为 `UserCFModel` 类。训练过程首先拟合 Baseline 模型，然后构造用户-物品残差矩阵。与 ItemCF 的列向量表示物品不同，UserCF 使用每一行表示一个用户的残差向量，并计算用户之间的余弦相似度。为减少少量共同评分带来的偶然高相似，程序同样引入共同评分阈值和收缩系数：

Listing 31: UserCF 基于残差计算用户相似度

```

1 numerators = residual_matrix @ residual_matrix.T
2 common_counts = mask_matrix @ mask_matrix.T
3 valid = (denom > 1e-12) & (common_counts[user_idx] >= self.min_common)
4 sims[valid] = numerators[user_idx, valid] / denom[valid]
5 sims[valid] *= common_counts[user_idx, valid] / (
6     common_counts[user_idx, valid] + self.shrinkage
7 )

```

预测阶段，程序从已经评价过目标物品的用户中寻找与目标用户相似的邻居用户，并取相似度绝对值最大的前 `top_k` 个用户参与加权残差修正。UserCF 的核心结构占用主要来自用户相似邻居表和物品到用户评分表。由于本数据集中用户数只有 598，远少于 9077 个物品，因此 UserCF 的相似度表占用低于 ItemCF；但从实验结果看，单独依赖用户相似性并不能取得最优 RMSE。因此，UserCF 在本实验中主要作为融合模型的补充信号，而不是最终单独使用的主模型。

6 Ensemble 融合算法详细说明

6.1 算法思想

课程课件在介绍 Hybrid Methods 时给出的核心思想是：可以实现两个或多个不同的推荐器，并将它们的预测结果进行组合，例如使用线性模型进行加权融合。该思想适用于单一推荐算法各有优势和局限的场景。对于本实验而言，ItemCF、Matrix Factorization 和 UserCF 都是在 Baseline 基础上进一步学习评分结构，但三者关注的信息并不完全相同。

ItemCF 更强调物品之间的局部相似关系。它根据用户已经评分过的相似物品，对目标物品的 Baseline 残差进行修正，因此当目标用户有足够相似邻居评分时，ItemCF 能较好地利用“喜欢相似物品，也可能喜欢该物品”的协同过滤信息。Matrix Factorization 则通过低维用户隐向量和物品隐向量学习整体潜在交互结构，它不局限于某几个近邻物品，而是尝试用较少潜在因素概括大量用户-物品评分模式。

UserCF 从另一个方向利用协同过滤信息。它不寻找目标物品的相似物品，而是寻找与目标用户相似的其他用户，再根据这些相似用户在目标物品上的残差评分进行修正。虽然 UserCF 单模型在本数据集上不一定优于 ItemCF，但它的误差来源与 ItemCF 和矩阵分解并不完全相同，因此可以作为融合模型中的补充预测信号。

由于这些方法的建模角度不同，它们的预测误差也可能具有互补性。若某些样本中 ItemCF 受邻居覆盖不足影响较大，矩阵分解的低维结构可能提供更平滑的估计；若某些样本中矩阵分解因低维表达较粗糙而预测不足，ItemCF 的局部相似物品残差又可能提供更直接的修正；若某些样本在用户侧存在较相近的评分模式，UserCF 则可以提供额外的用户相似性修正。因此，本实验先实现了 ItemCF 与 Matrix Factorization 的二模型线性融合：

$$\hat{r}_{ui}^{ens} = \alpha \hat{r}_{ui}^{ItemCF} + (1 - \alpha) \hat{r}_{ui}^{MF} \quad (24)$$

其中， α 表示 ItemCF 预测结果的权重， $1 - \alpha$ 表示矩阵分解预测结果的权重。当 $\alpha = 1$ 时，融合模型退化为单独 ItemCF；当 $\alpha = 0$ 时，融合模型退化为单独矩阵分解。该方法不引入新的用户属性、物品属性或外部内容特征，而是直接利用两个已训练推荐器的输出结果进行组合。

在进一步实验中，融合模型扩展为三模型线性融合：

$$\hat{r}_{ui}^{opt} = w_1 \hat{r}_{ui}^{ItemCF_b} + w_2 \hat{r}_{ui}^{MF} + w_3 \hat{r}_{ui}^{UserCF} \quad (25)$$

其中， $ItemCF_b$ 表示使用调优 Baseline 正则参数后构建残差的 ItemCF， $w_1 + w_2 + w_3 = 1$ 。该形式仍然保持线性融合的可解释性和较低过拟合风险，但可以同时利用物品相似、隐向量交互和用户相似三类信号。最终三模型权重不是人工指定，而是在 `Train.txt` 内部验证集上搜索得到。

6.2 程序结构

二模型融合在 `ensemble.py` 中实现为 `EnsembleModel` 类。该类内部同时保存一个 ItemCF 子模型和一个 Matrix Factorization 子模型，并通过 `itemcf_weight` 控制二者在最终预测中的比例。三模型优化融合在 `optimized_ensemble.py` 中实现为 `OptimizedEnsembleModel` 类，在二模型基础上增加 UserCF 子模型，并分别保存三类预测器的权重。主要结构如表 7 所示。

表 7: Ensemble 实现中的主要结构

结构或变量	作用
<code>itemcf</code>	保存 ItemCF 子模型，用于产生基于相似物品残差的预测分数。优化融合中该子模型使用调优 Baseline 残差。
<code>matrix_factorization</code>	保存 Matrix Factorization 子模型，用于产生基于隐向量点积的预测分数。
<code>usercf</code>	保存 UserCF 子模型，用于产生基于相似用户残差的预测分数。
<code>itemcf_weight</code> 、 <code>mf_weight</code> 与 <code>usercf_weight</code>	保存三模型融合权重。二模型融合中只使用 <code>itemcf_weight</code> ，矩阵分解权重为 $1 - \alpha$ 。
<code>min_rating</code> 与 <code>max_rating</code>	保存评分边界，用于后续预测结果截断。

二模型融合中较优权重为 `itemcf_weight=0.55`，即 55% ItemCF 预测和 45% 矩阵分解预测。进一步优化后，三模型融合采用 `itemcf_weight=0.43`、`mf_weight=0.45`、`usercf_weight=0.12`。这些权重不是人工凭直觉指定，而是在交叉验证中先进行粗粒度搜索，再在较优区间内细粒度搜索后确定的。权重搜索过程和实验结果将在实验分析部分进一步说明。

6.3 子模型训练过程

融合模型训练阶段不直接学习新的用户向量、物品向量或相似度矩阵，而是分别训练各个子模型。代码中 `fit` 函数依次调用 ItemCF、Matrix Factorization 和 UserCF 的训练过程：

Listing 32: 三模型融合的子模型训练过程

```

1 def fit(self, train_data):
2     self.itemcf.fit(train_data)
3     self.matrix_factorization.fit(train_data)
4     self.usercf.fit(train_data)
5     self.min_rating = self.itemcf.min_rating
6     self.max_rating = self.itemcf.max_rating
7     return self

```

这段代码说明，Ensemble 模型本身并不会改变各子模型的内部训练方式。ItemCF 仍然按前文方法训练 Baseline、构建残差矩阵并计算物品相似度；矩阵分解仍然按前文方法使用 Baseline 初始化偏置项，并通过 SGD 训练用户隐向量和物品隐向量；UserCF 则计算用户残差相似度并保存相似用户邻居。融合模型只是把训练好的推荐器作为组成模块，在预测阶段统一调用。

这种实现方式的优点是结构清晰，便于复用和比较。单独运行 ItemCF、Matrix Factorization 或 UserCF 时，三者仍然是完整模型；运行 Ensemble 时，则在不改动子模型内部逻辑的情况下组合多个模型输出。缺点是训练时间和核心结构占用会增加，因为每一折验证都需要分别训练并保存多个子模型。

6.4 融合预测过程

预测阶段，融合模型先分别调用各个子模型得到同一个用户-物品对的预测分数，再按照线性权重合成最终结果：

Listing 33: 三模型线性融合预测

```

1 itemcf_score = self.itemcf.predict(user_id, item_id)
2 mf_score = self.matrix_factorization.predict(user_id, item_id)
3 usercf_score = self.usercf.predict(user_id, item_id)
4 return (
5     self.itemcf_weight * itemcf_score
6     + self.mf_weight * mf_score
7     + self.usercf_weight * usercf_score
8 )

```

该代码直接对应三模型线性融合公式。线性加权的含义是：最终预测不完全相信某一个子模型，而是在多个模型之间做折中。当多个模型预测接近时，融合结果也接近它们的共同判断；当模型预测差异较大时，权重较高的模型对结果影响更大。

从模型选择角度看，线性融合还有一个重要优点：它的可解释性较强。权重 α 可以直接解释为 ItemCF 预测在最终结果中的占比，实验中也可以通过控制变量方式观察不同 α 对 RMSE 的影响。相比引入复杂的二层模型或非线性融合器，线性融合参数少、过拟合风险低，更适合本实验这种评分矩阵稀疏且样本规模有限的课程数据集。

6.5 冷启动与核心结构占用

Ensemble 的冷启动处理依赖各子模型自身的回退机制。ItemCF 在找不到可用相似邻居、用户未见或物品未见时，会回退到 Baseline 预测；矩阵分解在用户或物品未见时，会只使用已知偏置项和全局平均分；UserCF 在找不到相似用户或目标物品没有相似用户评分时，也会回退到 Baseline 预测。因此，融合模型即使遇到半冷启动用户-物品对，也能从各子模型中得到合法预测分数，再进行加权平均。

在核心结构占用方面，Ensemble 需要同时保存多个子模型。代码中的估计也直接将各子模型的核心结构相加：

Listing 34: Ensemble 核心结构占用估计

```

1 def approximate_size_bytes(self):
2     size = sys.getsizeof(self)
3     size += self.itemcf.approximate_size_bytes()
4     size += self.matrix_factorization.approximate_size_bytes()
5     size += self.usercf.approximate_size_bytes()
6     return size

```

这意味着融合模型的核心结构占用接近各子模型占用之和。由于 ItemCF 需要保存大量物品-物品相似邻居，UserCF 需要保存用户相似邻居和物品评分倒排表，而矩阵分解只保存低维隐向量和偏置项，因此优化后的三模型融合核心结构占用高于原二模型融合。实验结果也验证了这一点：三模型融合获得了更低 RMSE，但训练时间和模型占用也进一步增加。

7 实验设计与结果分析

7.1 验证方法与评价指标

由于 Test.txt 中不包含真实评分，本实验不能直接在测试集上计算 RMSE。为比较不同推荐算法的预测效果，实验只在 Train.txt 内部构造验证集，并使用验证集真实评分评价模型。主实验采用

5 折交叉验证：将训练集中每个用户的评分记录打乱后轮转分配到 5 个折中，每次取其中 1 折作为验证集，其余 4 折作为训练集，最后统计 5 次验证结果的平均值和标准差。

本章实验按照“基础比较-参数调优-融合优化-最终提交”的顺序展开。首先，基础模型主实验比较 Baseline、ItemCF、Matrix Factorization 和二模型 Ensemble 的整体表现，用于判断不同算法类型在该数据集上的基本优劣。其次，参数调优实验分别考察 Baseline 迭代与正则参数、ItemCF 相似度与邻居规模参数、矩阵分解隐向量维度与正则化参数，以及融合权重对验证集 RMSE 和计算开销的影响。再次，优化模型综合对比将调优后的 ItemCF、矩阵分解、UserCF 和融合模型放在统一的多随机种子验证框架下比较。最后，最终提交阶段使用完整 `Train.txt` 重新训练选定模型，并对 `Test.txt` 中的未知用户-物品对输出预测评分。

需要说明的是，报告中出现的 4 维、8 维和 48 维 MF 对应不同实验阶段，并不是前后矛盾。基础模型主实验中的 MF 使用 4 维残差矩阵分解，目的是在较低复杂度下与 Baseline、ItemCF 和二模型 Ensemble 做初始比较；后续调参先发现 8 维残差矩阵分解优于低维配置，进一步扩展维度后又发现 48 维 MF 在融合模型中取得更低验证误差。因此最终提交使用的是 ItemCF、48 维 Matrix Factorization 与 UserCF 的加权融合，即 43% 调优 ItemCF、45% 调优 MF 和 12% UserCF。

5 折交叉验证并不是题目强制要求，而是本实验在评估可靠性和计算开销之间做出的选择。为避免仅凭经验确定折数，实验使用原二模型 Ensemble 作为代表模型，对 3 折、5 折、6 折、8 折和 10 折交叉验证进行了对比。这里选用原二模型 Ensemble 的目的，是在一个效果较好、计算复杂度适中的模型上观察折数变化对评估稳定性和计算开销的影响；它不用于替代后文最终三模型融合的性能结论。该实验保持模型参数不变，只改变交叉验证折数，统计平均 RMSE、RMSE 标准差、单折平均训练时间以及完成整轮交叉验证所需的累计训练时间。结果如表 8 所示。

表 8: 不同交叉验证折数对 Ensemble 评估结果的影响

折数	平均 RMSE	RMSE 标准差	单折平均训练时间/s	累计训练时间/s	验证集规模
3	17.219074	0.070685	11.2757	33.8271	约 30285/折
5	16.952308	0.035910	14.0282	70.1411	约 18171/折
6	16.922238	0.166892	11.7238	70.3429	约 15142/折
8	16.876408	0.100109	17.7135	141.7078	约 11357/折
10	16.851493	0.152304	14.5277	145.2766	约 9085/折

表 8 给出了不同折数下的精确数值，图 7.1 进一步呈现平均 RMSE、标准差和训练时间之间的变化关系。

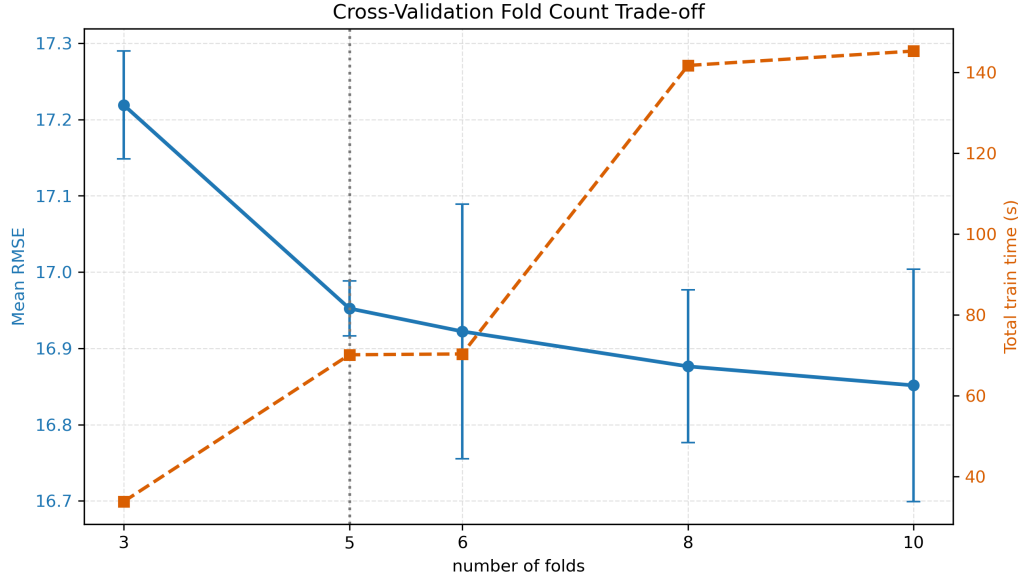


图 7.1: 交叉验证折数对评估结果与训练时间的影响

表 8 和图 7.1 表明, 3 折交叉验证的累计训练时间最短, 但每次只使用约三分之二的训练数据, 平均 RMSE 高于 5 折, 标准差也高于 5 折。表中的累计训练时间表示完成对应 K 折交叉验证所需的总时间, 即同一模型被训练 K 次后的耗时总和; 它不是最终模型在完整训练集上训练一次的时间。将折数从 5 增加到 6 时, 平均 RMSE 从 16.952308 小幅降至 16.922238, 但标准差升至 0.166892, 说明不同验证折之间波动明显变大; 继续增加到 8 折和 10 折后, 平均 RMSE 仍有小幅下降, 但累计训练时间约为 5 折的两倍, 且 RMSE 标准差分别为 0.100109 和 0.152304, 均高于 5 折。相比之下, 5 折交叉验证的平均 RMSE 明显优于 3 折, 标准差最低, 并且累计训练时间仍处于可接受范围。因此, 主实验采用 5 折交叉验证作为模型比较和参数选择的统一验证方案。

这种划分方式比完全随机打散更适合本实验数据。一方面, 它仍然保证每一折都覆盖不同用户的评分记录; 另一方面, 它减少了某一验证折中出现大量完全未见用户的概率, 使验证结果更接近最终评分预测任务中的普通泛化情形。代码中的折划分逻辑如代码片段 35 所示。

Listing 35: 按用户构造 5 折验证集

```

1 folds = build_user_k_folds(full_train, n_splits=N_FOLDS, seed=SEED)
2 for valid_index in range(N_FOLDS):
3     valid_data = folds[valid_index]
4     train_data = [
5         record
6         for fold_index, fold in enumerate(folds)
7         if fold_index != valid_index
8         for record in fold
9     ]

```

每一折训练完成后, 程序对验证集中的用户-物品对进行预测, 并将预测评分限制在训练集评分范围内, 然后计算 RMSE。RMSE 的定义为:

$$RMSE = \sqrt{\frac{1}{|V|} \sum_{(u,i,r_{ui}) \in V} (r_{ui} - \hat{r}_{ui})^2} \quad (26)$$

其中, V 表示验证集, r_{ui} 为真实评分, $\hat{r}_{ui}$ 为模型预测评分。RMSE 越小, 说明预测评分与真实评分之间的平均偏差越小。

除 RMSE 外, 实验还重点统计训练时间和核心模型持久化结构近似占用, 以对应任务书中对算法效果、训练时间和空间消耗的要求。训练时间使用 `model.fit` 前后的计时差, 空间占用由各模型的 `approximate_size_bytes` 函数估计核心数据结构大小。实验程序也会记录预测阶段耗时, 主要用于辅助检查程序运行状态; 但由于本任务的主要性能要求是训练时间和空间消耗, 后文表格统一报告训练时间而不把预测时间作为主要比较指标。需要说明的是, 空间指标不是程序运行时的真实峰值内存, 而是对模型训练完成后需要长期保存的主要数组、映射表、偏置表和相似邻居表进行估算, 不包含 Python 解释器、NumPy 临时矩阵、垃圾回收状态和操作系统额外开销。对应代码如代码片段 36 所示。

Listing 36: 训练时间、空间占用与 RMSE 统计

```

1 train_start = time.perf_counter()
2 model.fit(train_data)
3 train_seconds = time.perf_counter() - train_start
4
5 predict_start = time.perf_counter()
6 predictions = model.batch_predict(valid_pairs)
7 predict_seconds = time.perf_counter() - predict_start
8
9 y_pred = [
10     clip_rating(score, rating_min, rating_max)
11     for _, _, score in predictions
12 ]
13 fold_rmse.append(rmse(y_true, y_pred))
14 memory_bytes_list.append(float(model.approximate_size_bytes()))

```

本实验运行环境为 Python 3.12.3, 运行系统为 Linux WSL2。由于训练时间会受到机器性能、后台负载和解释器状态影响, 时间指标主要用于比较同一环境下不同算法之间的相对开销。

7.2 基础模型主实验

主实验比较 Baseline、ItemCF、Matrix Factorization 和 Ensemble 四种算法。Baseline 使用全局平均分、用户偏置和物品偏置; ItemCF 在 Baseline 残差上计算物品相似度, 并在预测阶段使用相似物品残差修正基础分; Matrix Factorization 使用带偏置项的隐向量点积模型, 并通过 SGD 训练参数; Ensemble 则对 ItemCF 与 Matrix Factorization 的预测结果进行线性融合。

本实验中, 5 折交叉验证的每折验证集规模分别为 18156、18179、18186、18165 和 18168 条评分。四种算法的实验结果如表 9 所示。

表 9: 多种推荐算法的 5 折交叉验证结果

算法	平均 RMSE	RMSE 标准差	平均训练时间/s	核心结构占用/MB
Baseline	17.524633	0.036371	0.4774	0.7462
ItemCF	17.112610	0.045719	4.9639	15.1663
Matrix Factorization	17.193329	0.027584	10.8121	0.9526
Ensemble	16.952308	0.035910	16.1105	16.1189

表 9 给出了四种算法的精确指标, 图 7.2 从准确率、训练时间和核心结构占用三个维度对结果进行对比。Baseline 的 RMSE 最高, 但训练时间和模型占用最低, 是 ItemCF、矩阵分解和融合模型共

同依赖的稳定基准。

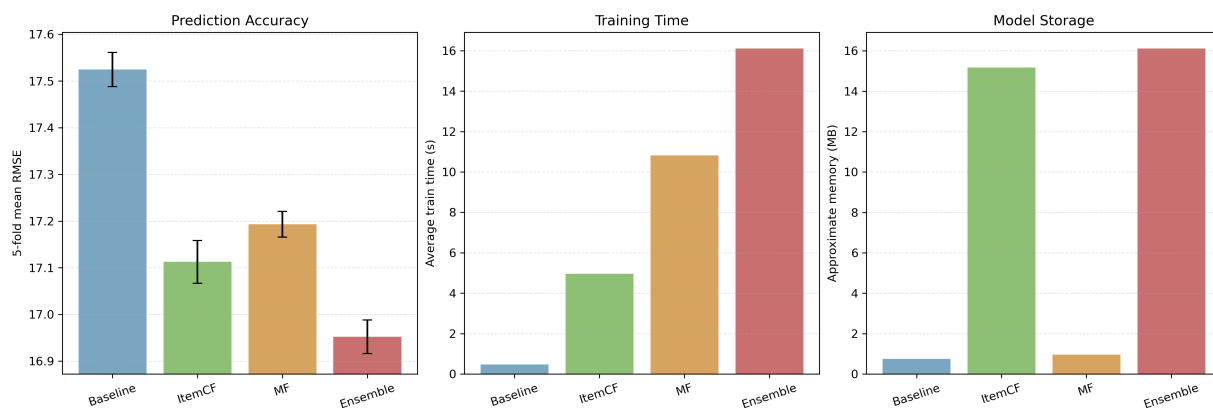


图 7.2: 四种推荐算法的准确率、训练时间与核心结构占用对比

从 RMSE 看, Ensemble 的平均 RMSE 最低, 为 16.952308; ItemCF 的平均 RMSE 为 17.112610; Matrix Factorization 的平均 RMSE 为 17.193329; Baseline 的平均 RMSE 为 17.524633。与 Baseline 相比, Ensemble 的 RMSE 降低了 0.572325, 约降低 3.27%; 与单独 ItemCF 相比, Ensemble 也进一步降低了 0.160302。该结果表明, ItemCF 和矩阵分解虽然都建立在 Baseline 之上, 但二者捕捉的误差结构并不完全相同: ItemCF 更依赖物品相似邻居, 矩阵分解则通过低维隐向量刻画整体潜在交互, 将二者按合适权重融合后可以减少单一模型的系统性偏差。

从稳定性看, Matrix Factorization 的 RMSE 标准差为 0.027584, 低于 Ensemble 的 0.035910 和 ItemCF 的 0.045719, 说明矩阵分解在不同验证折上的波动更小。Ensemble 的平均 RMSE 最低, 标准差介于矩阵分解和 ItemCF 之间, 表明融合后准确率得到提升, 同时波动也比单独 ItemCF 略小。

需要说明的是, 表 9 中 ItemCF 和 Ensemble 的核心结构占用采用的是紧凑邻居表实现后的估计值。最初实现中, ItemCF 使用嵌套 Python 字典保存相似邻居关系, 即 `item_id -> {neighbor_id: similarity}`。这种结构查找方便, 但每条相似关系都会带来字典、键和值对象等额外开销, 因此基础 ItemCF 的核心结构占用曾达到约 97.4959 MB, 二模型 Ensemble 约为 98.4486 MB。后续实现将每个物品的邻居编号和相似度分别保存为 NumPy 数组 `item_neighbor_ids` 与 `item_neighbor_sims`, 其中编号使用 `int32`, 相似度使用 `float32`, 并按相似度绝对值预排序。该修改不改变相似度计算和预测公式, 只改变邻居表的存储方式, 因此 RMSE 基本保持不变, 但基础 ItemCF 的核心结构占用降至 15.1663 MB, 二模型 Ensemble 降至 16.1189 MB。

从时间和核心结构占用看, Baseline 训练最快, 平均训练时间只有 0.4774 秒, 核心结构占用约 0.7462 MB, 适合作为稳健基准模型。ItemCF 的平均训练时间增加到 4.9639 秒, 核心结构占用约为 15.1663 MB, 主要原因是它需要构造残差矩阵并保存物品相似邻居。当前 ItemCF 配置中 `similarity_top_n=200`, 训练集共有 9077 个物品, 因此模型最多需要保存约 9077×200 条物品-物品相似关系。相比之下, Matrix Factorization 的平均训练时间为 10.8121 秒, 核心结构占用只有约 0.9526 MB, 因为它主要保存 4 维用户隐向量、4 维物品隐向量以及偏置项, 参数规模近似为 $598 \times 4 + 9077 \times 4$, 远小于 ItemCF 的相似邻居表。Ensemble 同时保存 ItemCF 和矩阵分解两个子模型, 因此核心结构占用约为 16.1189 MB, 训练时间也增加到 16.1105 秒。上述基础模型对比说明融合方法具有较好潜力, 但仍需要进一步分析各子模型参数和融合权重。综合来看, 最终模型选择存在准确率与开销的权衡: 融合模型准确率最高, 但训练时间最长; 矩阵分解准确率略低, 但模型更紧凑。

7.3 参数调优实验

7.3.1 Baseline 迭代轮数与正则参数

Baseline 模型的可调参数主要包括交替迭代轮数 `n_iters`、用户偏置正则化系数 `reg_user` 和物品偏置正则化系数 `reg_item`。迭代轮数决定用户偏置与物品偏置相互校正的充分程度；正则化系数决定偏置项向 0 收缩的强度。若迭代轮数过少，用户偏置和物品偏置尚未稳定；若迭代轮数过多，RMSE 改善很小但训练时间继续增加。若正则化过强，用户和物品的个性化偏置会被过度压缩；若正则化过弱，长尾用户或长尾物品可能因为少量评分产生过大的偏置。

为了说明代码中 `n_iters=15` 的选择依据，实验在其他参数保持不变的条件下，对不同迭代轮数进行了 5 折交叉验证。结果如表 10 所示。

表 10: Baseline 迭代轮数对验证 RMSE 和训练时间的影响

迭代轮数	平均 RMSE	RMSE 标准差	平均训练时间/s
1	17.678842	0.043154	0.124030
2	17.556794	0.038237	0.152339
3	17.538099	0.037238	0.217782
5	17.528045	0.036631	0.277824
8	17.525174	0.036415	0.441819
10	17.524792	0.036384	0.525818
15	17.524633	0.036371	0.691220
20	17.524625	0.036370	0.921001
30	17.524624	0.036370	1.308788

从表 10 可以看出，迭代轮数从 1 增加到 5 时，RMSE 下降较明显，说明用户偏置和物品偏置在前几轮中仍在快速互相修正；从 10 轮增加到 15 轮时，RMSE 只从 17.524792 降至 17.524633，改善已经很小；继续增加到 20 轮和 30 轮后，RMSE 几乎不再变化，但训练时间仍持续增加。因此，本实验保留 15 轮作为折中设置：它能够基本达到收敛状态，同时避免无意义地增加训练时间。

在正则参数方面，默认配置 `reg_user=10`, `reg_item=15` 的 5 折平均 RMSE 为 17.524633；将其调整为 `reg_user=6`, `reg_item=10` 后，平均 RMSE 降至 17.417113。该结果说明默认正则略强，适当减小正则可以保留更多用户和物品偏置信息。因此，后续调优 ItemCF 在构造残差时采用该 Baseline 配置，使相似度模型建立在更准确的基础偏置估计之上。

7.3.2 ItemCF 参数调优

ItemCF 的核心参数会影响相似度可靠性、邻居覆盖率和计算开销。参数实验在 `Train.txt` 内部固定划分 72901 条记录作为训练集、17953 条记录作为验证集，并分别调整收缩系数、共同评分阈值、预测邻居数、相似邻居保留数和分块大小。为使曲线能够体现变化趋势，而不是只比较少数几个离散点，每个参数设置 5 到 7 个取值进行控制变量扫描。参数实验代码中保留了多组配置，如代码片段 37 所示。

其中，收缩系数 `shrinkage` 用于修正物品相似度的可信程度。ItemCF 中两个物品的相似度通常由共同评分用户计算得到，但如果两个物品只有很少共同评分用户，即使余弦相似度很高，也可能只是少量样本造成的偶然结果。收缩系数的作用就是在共同评分用户数较少时，将相似度向 0 压缩；当共同评分用户数较多时，说明相似度有更多评分证据支撑，收缩影响会变小。

本实验中的相似度修正可以理解为：

$$sim'(i, j) = sim(i, j) \cdot \frac{c_{ij}}{c_{ij} + \gamma} \quad (27)$$

其中, $sim(i, j)$ 表示原始余弦相似度, c_{ij} 表示物品 i 和物品 j 的共同评分用户数, γ 就是收缩系数。若 c_{ij} 很小, 则 $\frac{c_{ij}}{c_{ij} + \gamma}$ 明显小于 1, 相似度会被压低; 若 c_{ij} 较大, 该比例接近 1, 修正后的相似度更接近原始余弦相似度。因此, **shrinkage** 并不是直接缩小预测评分, 而是降低低证据相似关系在预测中的影响。

Listing 37: ItemCF 参数实验配置片段

```

1 configs = [
2     ("shrinkage", [0, 5, 10, 20, 30, 50, 80]),
3     ("min_common", [1, 2, 3, 4, 5]),
4     ("top_k", [5, 10, 20, 30, 40, 60, 80]),
5     ("similarity_top_n", [40, 60, 80, 100, 120, 160, 200]),
6     ("block_size", [64, 128, 256, 512, 1024]),
7 ]

```

为了使参数分析符合控制变量原则, 表 11 以 $top_k=20$ 、 $similarity_top_n=80$ 、 $shrinkage=10.0$ 、 $min_common=2$ 、 $block_size=256$ 作为基准配置。除“联合配置复核”外, 其余每组实验都只相对于基准配置改变一个参数。也就是说, 当分析 **shrinkage** 时, top_k 、 $similarity_top_n$ 、 min_common 和 $block_size$ 均保持不变; 当分析 top_k 时, 其他参数也保持为基准值。这样可以尽量把 RMSE、核心结构占用或训练时间的变化归因到当前被调整的参数上。

表 11: ItemCF 参数敏感性实验摘要 (控制变量)

变化参数	取值范围	最优取值	最低 RMSE	主要现象
shrinkage	0-80	50	17.396125	RMSE 先明显下降, 随后趋于平缓, 空间基本不变
min_common	1-5	3	17.488584	阈值过低噪声多, 过高覆盖不足, 空间随阈值增大而下降
top_k	5-80	40	17.502089	从 5 到 20 提升明显, 之后边际收益很小
similarity_top_n	40-200	200	17.256936	保留更多相似邻居可降低 RMSE, 但空间增至 15.1812 MB
block_size	64-1024	不影响 RMSE	17.508176	主要影响训练时间, 不改变最终邻居集合

表 11 给出了每组控制变量实验的摘要, 完整数值保存于 `itemcf_parameter_results.csv`。参数趋势按照含义拆分为三组: 图 7.3 分析相似度可靠性参数, 图 7.4 分析邻居规模参数, 图 7.5 分析工程参数 **block_size** 对训练时间的影响。为避免图形信息过载, 只有空间占用随参数明显变化的 **similarity_top_n** 图保留内存曲线; **shrinkage**、**top_k** 和 **block_size** 等空间基本不变的参数则只展示 RMSE 或训练时间趋势。

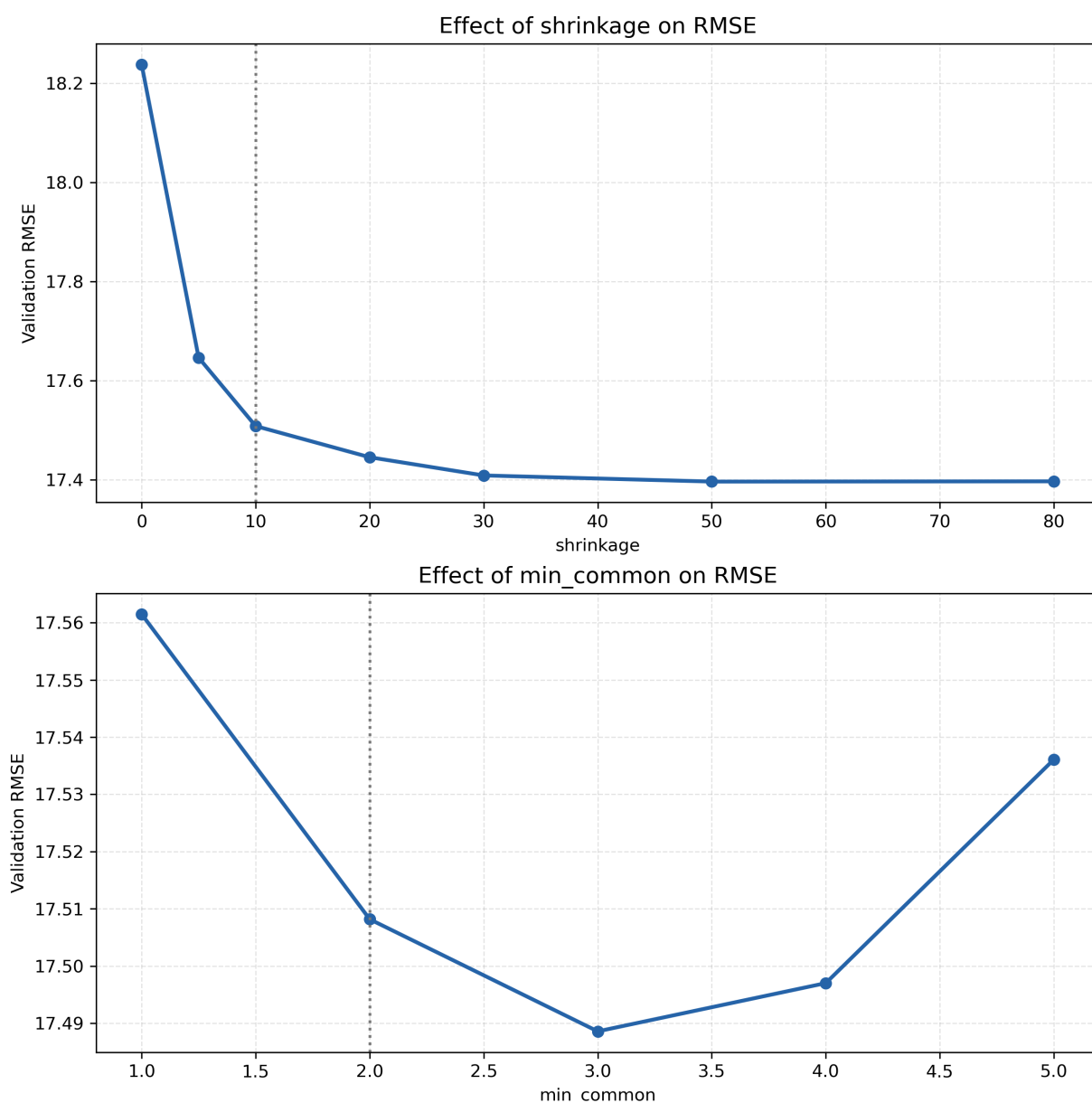


图 7.3: ItemCF 相似度可靠性参数趋势分析

图 7.3 反映相似度可靠性参数的影响。实验结果表明，收缩系数对 ItemCF 的效果影响明显。在保持其他参数不变时，不使用收缩，即 `shrinkage=0`，验证 RMSE 为 18.238219，明显差于基准配置的 17.508176；当收缩系数逐步提高到 30 和 50 时，RMSE 分别降至 17.408523 和 17.396125；继续提高到 80 后，RMSE 基本不再下降。该现象与数据集集中的长尾物品有关：如果直接采用少量共同用户形成的余弦相似度，容易产生偶然的高相似关系。加入 `shrinkage` 后，共同评分用户数较少的相似度会被压低，预测结果更稳定，但收缩过大后收益趋于平缓。

共同评分阈值 `min_common` 体现了相似度证据数量要求。实验中 `min_common=1` 时核心结构占用约为 13.0119 MB，且 RMSE 为 17.561486，说明过低阈值会保留较多证据不足的相似关系；提高到 3 时，RMSE 降至 17.488584，同时核心结构占用下降到约 9.8169 MB；继续提高到 5 后，RMSE 又升至 17.536086，说明阈值过高会丢弃过多可用邻居。因此，`min_common` 需要在过滤噪声和保留覆盖率之间折中。

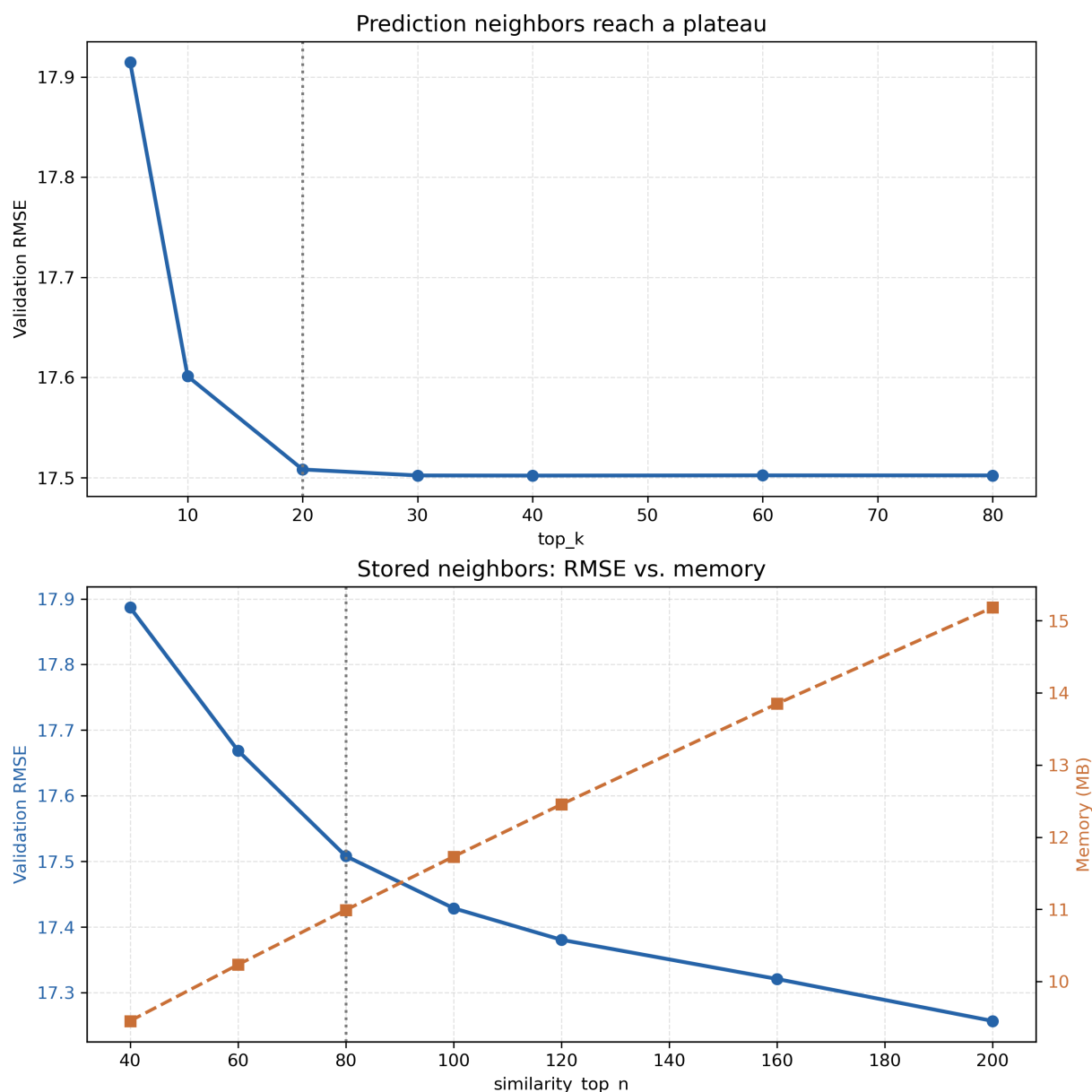


图 7.4: ItemCF 邻居规模参数趋势分析

图 7.4 反映预测阶段能够利用的邻居规模。在保持每个物品最多保留 80 个相似邻居时， $\text{top_k}=5$ 的 RMSE 为 17.914845，说明预测阶段使用的邻居过少时，可利用的个性化残差信息不足；当 top_k 增加到 20 后，RMSE 降至 17.508176；继续增加到 40、60 和 80 后，RMSE 基本稳定在 17.502 左右，说明仅增加预测阶段邻居数量的边际收益很小。

相比之下，单独将 similarity_top_n 从 40 提高到 200 时，RMSE 从 17.886949 持续降至 17.256936，但核心结构占用也从约 9.4550 MB 增加到约 15.1812 MB。在本数据集上，保留更多物品相似邻居比单纯增加预测阶段的 top_k 更有效，因为它扩大了预测时可以被命中的候选相似物品集合；但该参数也会直接扩大物品-物品相似表，因此仍然带来模型占用代价。与嵌套字典存储相比，紧凑数组存储显著降低了该参数带来的绝对空间开销。

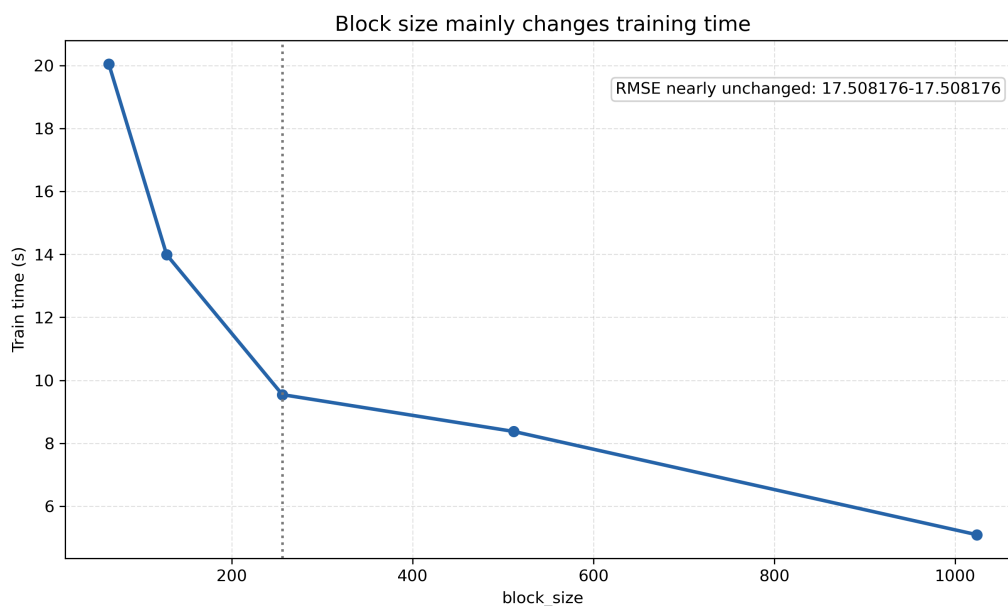


图 7.5: ItemCF block_size 对训练时间的影响

图 7.5 展示了分块大小对训练效率的影响。分块大小主要影响训练阶段的临时矩阵计算方式，不改变最终保留的相似邻居集合。在实验中，`block_size=64` 到 `block_size=1024` 的 RMSE 都约为 17.508176，几乎没有变化；但训练时间从约 20.0519 秒下降到约 5.0976 秒。因此，`block_size` 更多是工程性能参数，而不是模型精度参数。

需要特别说明的是，`block_size=1024` 在该控制变量实验中训练时间较短，但它只说明更大的分块可以减少循环管理开销，并不代表模型预测效果发生变化。主实验保留 `block_size=256`，是为了与参数实验的基准配置保持一致，使基础模型比较和单参数分析具有清晰口径；扩展优化阶段的 ItemCF 则采用更大的分块配置以提高运行效率。由于本文报告的空间指标是模型训练完成后的核心结构占用，而 `block_size` 主要影响训练过程中的临时矩阵规模和计算组织方式，因此它不会显著改变表中的持久化结构占用。

在单参数控制变量实验之外，本文只把联合参数结果作为候选配置复核，而不把它解释为完整网格搜索。原因是 `top_k`、`similarity_top_n` 和 `shrinkage` 的组合数量较多，如果逐一枚举会显著增加实验成本，也会使报告重点从算法比较转向工程调参。因此，这里仅沿着单参数实验已经显示出的方向进行少量验证：适当增大候选相似邻居数量，并提高对低证据相似度的收缩强度。复核结果显示，`top_k=40`，`similarity_top_n=120`，`shrinkage=10` 的 5 折平均 RMSE 为 17.321676；提高到 `similarity_top_n=200` 后，RMSE 降至 17.178714；进一步采用 `shrinkage=50` 后，RMSE 降至 17.112610。该结果说明联合配置与前面单参数趋势基本一致，但它只是最终参数选择的补充依据，而不是对全部参数空间的穷尽证明。基于这一复核，ItemCF 在主实验中采用 `top_k=40`、`similarity_top_n=200`、`shrinkage=50.0`、`min_common=2`、`block_size=256`。

在扩展探索中，实验继续沿着“提高候选邻居覆盖率、加强低证据相似度收缩”的方向验证更大的相似邻居保留数。结果表明，`top_k=40`，`similarity_top_n=320`，`shrinkage=120` 在相同 5 折划分上的平均 RMSE 降至 17.043444，低于主实验 ItemCF 的 17.112610。同时，Baseline 正则参数也会影响残差质量：将 ItemCF 内部 Baseline 的 `reg_user` 和 `reg_item` 从默认的 10、15 调整为 6、10 后，多随机种子验证中的调优 ItemCF 平均 RMSE 为 17.044970。该结果说明，ItemCF 的改进不仅来自相似邻居数量增加，也来自更合适的基础残差建模。由于该配置需要保存更多物品相似邻居，模型占用仍然高于单独矩阵分解；但在当前紧凑数组存储实现下，调优 ItemCF 的核心结构占用约为 18.8308

MB，没有继续保持在嵌套字典存储下的百 MB 量级。因此后续最终模型选择需要同时考虑准确率、训练时间和模型占用。

7.3.3 Matrix Factorization 参数调优

矩阵分解中的隐向量维度 `n_factors`、学习率 `learning_rate` 和正则化系数 `reg` 都是预设超参数。为便于在表格中简写，后文配置名中的 `d` 表示隐向量维度，`lr` 表示 SGD 学习率，`reg` 表示正则化系数。例如，`8d-lr0.002-reg0.3` 表示使用 8 维用户/物品隐向量，学习率为 0.002，正则化系数为 0.3。维度过低时，模型表达能力不足；维度过高时，参数数量增加，容易在稀疏数据上过拟合。学习率过大时，SGD 可能在最优解附近震荡甚至偏离；学习率过小时，训练过程收敛较慢。正则化过弱时，长尾用户和长尾物品的向量容易被少量评分牵引得过大；正则化过强时，模型又可能退化为接近 Baseline 的简单预测。

最初实验使用较常见的 5、10、20、40 和 80 维配置进行固定验证集测试，但结果并不理想，初始扫描结果如表 12 所示。其主要原因是该数据集虽然有 90854 条评分，但物品数达到 9077，评分矩阵稀疏度约为 0.016738，许多物品的评分样本较少。高维隐向量为每个长尾物品引入更多参数，反而容易学习到验证集不能泛化的噪声。根据这一现象，调参方向调整为：先用 Baseline 初始化偏置项，再使用低维隐向量、小学习率和较强正则化来学习残差交互。

表 12: 矩阵分解初始高维扫描结果

隐向量维度	RMSE	训练时间/s	核心结构/MB
5	18.854516	19.4348	0.9880
10	20.266418	18.9772	1.1603
20	20.916104	19.8219	1.5048
40	20.534987	20.1135	2.1938
80	36.535476	19.7031	3.5718

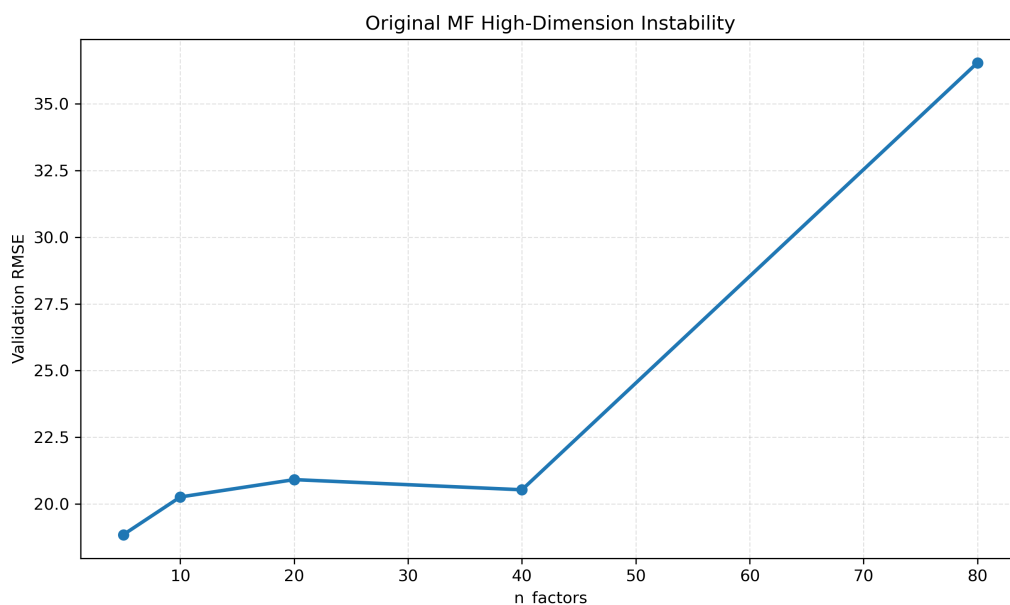


图 7.6: Matrix Factorization 原始训练方式的高维失稳现象

从表 12 和图 7.6 可以看出，直接增加隐向量维度并没有带来更低误差。5 维配置的 RMSE 已达到 18.854516，明显高于 Baseline；当维度增至 80 时，RMSE 上升到 36.535476，说明该训练方式下

高维参数出现了严重不稳定。因此，后续实现转向更稳健的残差矩阵分解：先用 Baseline 解释全局平均分和偏置项，再让隐向量只学习剩余的用户-物品交互。

为了避免把不同实验目标混在一起，矩阵分解调参分为两个阶段。第一阶段是低维参数预筛选，主要验证 Baseline 初始化、固定偏置、小学习率和较强正则化是否能稳定降低 RMSE；这一阶段会同时比较少量维度、学习率和正则化组合，因此它不是严格的单一变量维度扫描。第二阶段在第一阶段较稳定的训练设置基础上固定学习率和正则化，再单独扩展隐向量维度，用于判断更高维度是否仍然有效。低维预筛选结果如表 13 所示。

表 13: 矩阵分解低维残差模型参数预筛选结果

配置	维度	学习率	正则化	平均 RMSE	标准差	核心结构/MB
1d-lr0.003-reg0.2	1	0.003	0.2	17.325007	0.130953	0.8494
1d-lr0.003-reg0.1	1	0.003	0.1	17.313125	0.075628	0.8494
1d-lr0.002-reg0.2	1	0.002	0.2	17.357040	0.084024	0.8494
1d-lr0.004-reg0.2	1	0.004	0.2	17.356921	0.163852	0.8494
2d-lr0.002-reg0.2	2	0.002	0.2	17.244331	0.079771	0.8838
3d-lr0.002-reg0.2	3	0.002	0.2	17.216809	0.048267	0.9182
4d-lr0.002-reg0.2	4	0.002	0.2	17.193329	0.027584	0.9526
5d-lr0.002-reg0.3	5	0.002	0.3	17.167454	0.056961	0.9870
8d-lr0.002-reg0.3	8	0.002	0.3	17.129505	0.039504	1.0902

表 13 给出了低维残差矩阵分解的主要预筛选结果。表中前几行在 1 维模型下比较学习率和正则化强度，用于判断 SGD 更新步长和约束强度是否稳定；随后在较稳健的学习率和正则化设置附近逐步增加维度，观察模型表达能力提升后 RMSE 是否继续下降。因此，该表的作用是确定“残差训练方式是否有效，以及哪些参数范围值得继续扩展”，而不是最终的完整维度搜索。为避免多张子图并排后信息过密，矩阵分解参数趋势拆分为三张单独图展示。图 7.7 汇总采用 Baseline 初始化和固定偏置后的代表候选配置，用于比较精度提升、RMSE 波动和核心结构占用之间的关系；图 7.8 展示 2 维残差模型附近的局部参数搜索结果。

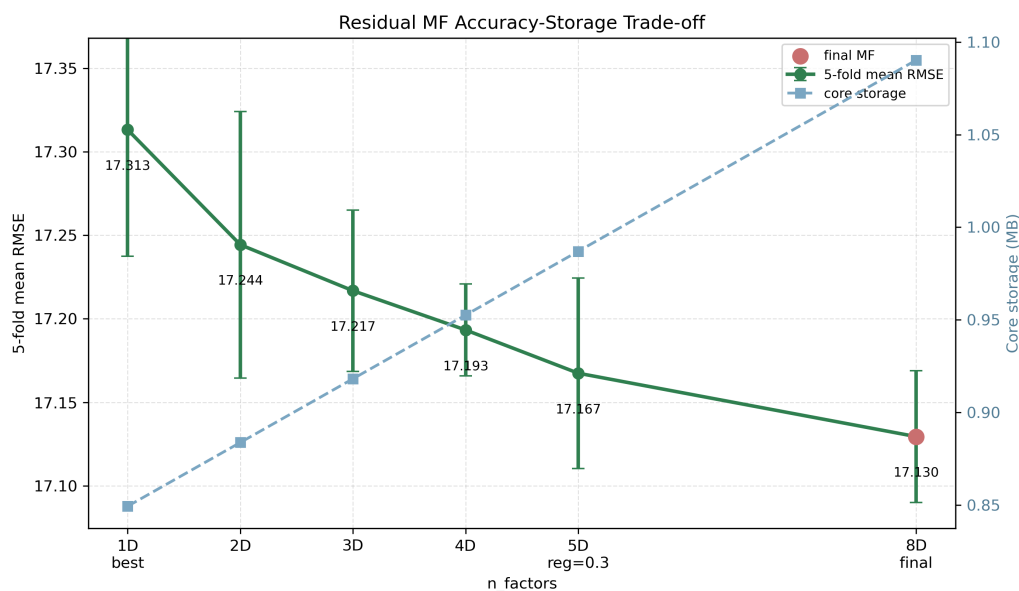


图 7.7: 残差矩阵分解的精度与空间折中

图 7.7 表明，改为 Baseline 初始化并固定偏置后，残差矩阵分解明显优于最初的高维直接训练方式。1 维代表配置已经可以达到 17.313125 的平均 RMSE，说明在 Baseline 之后，数据中仍存在可以

用少量潜在因素解释的剩余交互结构。随着维度增加到 2、3、4 维，模型能够表达更多用户-物品残差交互，平均 RMSE 逐步下降；继续在较强正则化下使用 5 维和 8 维后，RMSE 分别降至 17.167454 和 17.129505。图中的误差线表示 5 折交叉验证的 RMSE 标准差，右轴表示核心结构占用，可以看出维度增加会带来一定空间增长，但矩阵分解的核心结构仍保持在约 1 MB 量级，远低于需要保存相似邻居表的协同过滤模型。

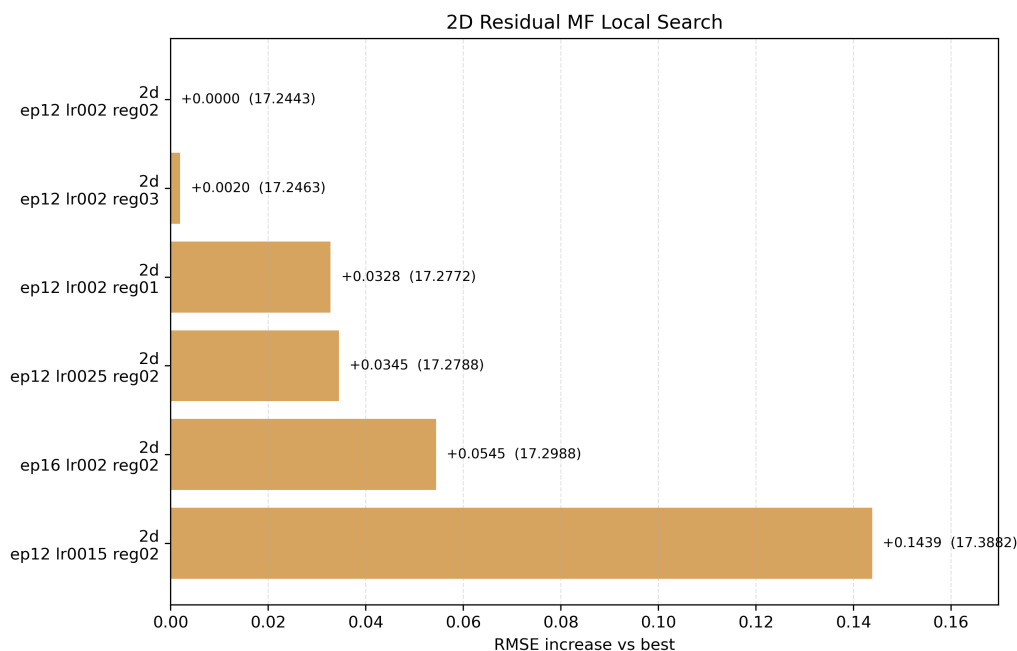


图 7.8: 2 维残差矩阵分解局部参数搜索的相对 RMSE 增量

图 7.8 不再直接绘制原始 RMSE，而是展示各配置相对于最优配置的 RMSE 增量。这样可以更清楚地观察局部参数变化带来的细微差异：学习率、正则化和训练轮数的变化虽然不会造成数量级差异，但仍会影响低维残差模型表现。因此，矩阵分解调参不能只看隐向量维度，也需要同时控制优化过程和正则约束。

在确认 Baseline 初始化和冻结偏置的有效性后，实验进入第二阶段：固定训练方式和主要优化参数，只考察隐向量维度的影响。具体来说，本阶段统一采用 `learning_rate=0.002`、`reg=0.3`、`init_std=0.01` 和 `freeze_bias=True`，并将隐向量维度从 8 扩展至 96。这里的 8 维同时是上一阶段低维预筛选中的较优点，也是高维扩展扫描的起点，因此两张表中都保留 8 维结果作为衔接基准。实验结果如表 14 所示。

表 14: 残差矩阵分解固定参数后的维度扩展扫描结果

隐向量维度	平均 RMSE	标准差
8	17.129505	0.039504
12	17.076949	—
16	17.062732	—
20	17.046174	0.042569
24	17.019312	0.032553
32	16.997325	0.044230
48	16.981150	0.039255
64	16.951135	0.037636
96	16.940853	0.033469

从表 14 可以看出，在 Baseline 初始化和冻结偏置条件下，增加残差隐向量维度在 8 至 96 的范围内仍能稳定降低验证误差，与表 12 中初始训练方式的失控行为截然不同。这是因为偏置项已先行解释了全局均值、用户宽严和物品热度等系统性信号，残差矩阵分解只需学习少量剩余的个性化交互结构；较强正则化和小初始化幅度同时抑制了高维嵌入在稀疏长尾物品上的过拟合倾向。因此，早期高维扫描的不稳定并非源于“高维必然过拟合”，而是源于当时缺乏 Baseline 初始化、冻结偏置和较强正则化等配置支撑。

然而，从 48 维到 96 维，平均 RMSE 仅从 16.981150 降至 16.940853，改善幅度约为 0.040，明显小于 8 维到 48 维阶段约 0.148 的提升。这表明残差交互结构在 48 维附近已得到较充分刻画，继续增加维度进入边际收益快速递减阶段。综合精度与训练效率，最终融合中采用 **48 维**残差矩阵分解作为子模型。矩阵分解的核心结构占用仍远低于 ItemCF 的相似邻居表，因此适合作为融合模型中的紧凑补充。最终使用的残差矩阵分解配置如代码片段 38 所示。

Listing 38: 最终融合中矩阵分解子模型的参数配置

```
1 MatrixFactorizationModel(  
2     n_factors=48,  
3     n_epochs=12,  
4     learning_rate=0.002,  
5     reg=0.3,  
6     reg_bias=0.05,  
7     init_std=0.01,  
8     init_with_baseline=True,  
9     freeze_bias=True,  
10    seed=SEED,  
11 )
```

其中，`init_with_baseline=True` 表示先训练 Baseline 并复制其全局平均分、用户偏置和物品偏置；`freeze_bias=True` 表示后续 SGD 中固定这些偏置项，只训练用户隐向量和物品隐向量。这样可以避免偏置项和隐向量重复解释同一部分评分误差，使矩阵分解更集中地学习 Baseline 之后的残差结构。

7.3.4 融合权重调优

在算法详细说明部分已经说明，Ensemble 使用 α 控制 ItemCF 和 Matrix Factorization 的预测占比。为确定该权重，实验在相同的 5 折交叉验证划分上进行两阶段搜索。第一阶段按 $\alpha = 0.0, 0.1, \dots, 1.0$ 做粗粒度扫描，用于判断较优权重大致范围；第二阶段在 $\alpha = 0.45$ 到 $\alpha = 0.65$ 之间按 0.01 步长细扫，用于观察最优点附近的变化趋势。

该实验只改变融合权重，不改变两个子模型的内部参数。ItemCF 使用前文确定的最终配置，矩阵分解仍使用 4 维隐向量、小学习率和较强正则化配置。为了避免对每个权重重复训练子模型，实验程序在每一折中先分别训练一次 ItemCF 和矩阵分解，缓存二者在验证集上的预测分数，然后对不同 α 进行线性组合并计算 RMSE。这样可以把 RMSE 的变化主要归因于融合比例本身。权重实验结果如表 15 所示。

表 15: Ensemble 融合权重实验结果

ItemCF 权重 α	MF 权重 $1 - \alpha$	平均 RMSE	RMSE 标准差
0.00	1.00	17.193329	0.027584
0.30	0.70	17.002388	0.030403
0.40	0.60	16.970410	0.032299
0.50	0.50	16.954338	0.034668
0.53	0.47	16.952643	0.035402
0.54	0.46	16.952394	0.035656
0.55	0.45	16.952308	0.035910
0.56	0.44	16.952382	0.036166
0.57	0.43	16.952610	0.036430
0.60	0.40	16.954244	0.037246
0.70	0.30	16.970275	0.039937
1.00	0.00	17.112610	0.045719

表 15 给出了部分代表性权重的精确数值，图 7.9 展示了粗粒度搜索和最优区间细扫的完整趋势。

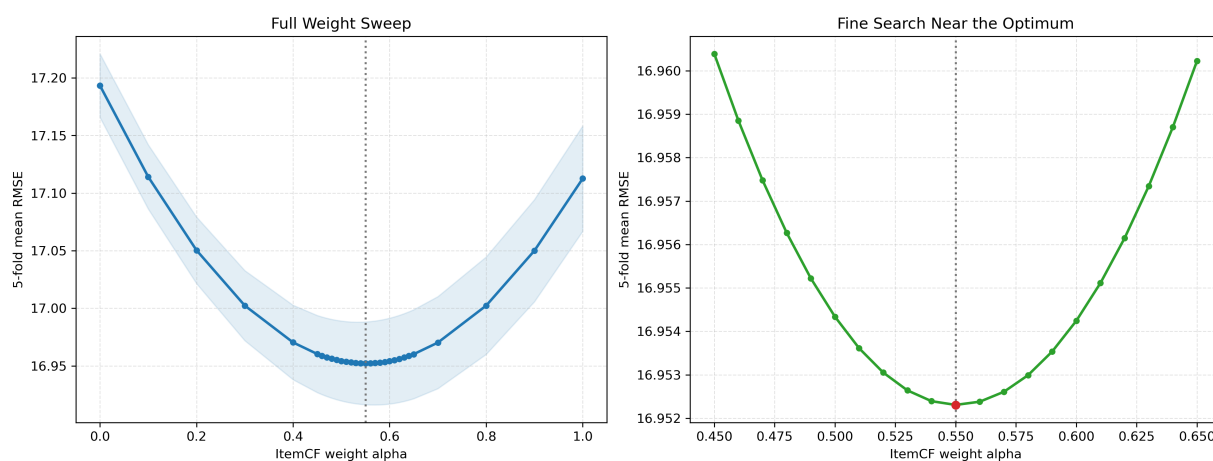


图 7.9: Ensemble 融合权重控制变量趋势分析

表 15 和图 7.9 表明，单独矩阵分解对应 $\alpha = 0$ ，平均 RMSE 为 17.193329；单独 ItemCF 对应 $\alpha = 1$ ，平均 RMSE 为 17.112610。粗粒度搜索中，当 ItemCF 权重从 0 增加到 0.5 和 0.6 附近时，RMSE 明显下降，说明两种模型的预测结果具有互补性。继续提高 ItemCF 权重后，RMSE 反而上升，说明并不是 ItemCF 权重越大越好。

进一步细扫发现， $\alpha = 0.53$ 到 $\alpha = 0.57$ 的平均 RMSE 都低于 16.953，最优区域较平坦。最低值出现在 $\alpha = 0.55$ ，平均 RMSE 为 16.952308，略低于 $\alpha = 0.60$ 时的 16.954244。虽然这次细调带来的提升只有约 0.001936，但融合权重仍然是可调参数，并且较细粒度的参数搜索可以在已有模型基础上获得小幅改进。因此，最终融合模型选择 $\alpha = 0.55$ ，即 55% ItemCF 预测和 45% 矩阵分解预测。

在扩展实验中，进一步把调优 ItemCF、调优矩阵分解和 UserCF 进行三模型融合。权重搜索仍然使用“先缓存各子模型预测、再扫描线性权重”的方式，避免重复训练子模型。粗粒度搜索显示，加入 UserCF 后虽然单模型 RMSE 较高，但在 10% 到 20% 的小权重范围内可以降低融合误差；随后在较优区域按 0.01 步长细扫。部分结果如表 16 所示。

表 16: 优化后三模型融合权重细扫结果

调优 ItemCF 权重	MF 权重	UserCF 权重	平均 RMSE	RMSE 标准差
0.43	0.45	0.12	16.837379	0.039802
0.44	0.45	0.11	16.837441	0.039778
0.42	0.45	0.13	16.837532	0.039829
0.44	0.44	0.12	16.837665	0.039774
0.45	0.45	0.10	16.837721	0.039736

表 16 表明，最优权重附近存在一个较平坦区域。最低平均 RMSE 出现在 0.43 : 0.45 : 0.12，但相邻权重组合的差异只有 10^{-4} 量级。这说明最终权重并不是依赖某一个尖锐的偶然点，而是在较宽的小范围内都能保持相近效果。最终优化融合采用 43% 调优 ItemCF、45% 矩阵分解和 12% UserCF 的线性组合。

7.4 优化模型综合对比

在完成基础模型比较、单模型参数调优和融合权重搜索后，实验进一步汇总较优候选模型的整体表现，重点比较调优 Baseline 正则后的 ItemCF、调优后的矩阵分解、UserCF 以及三模型融合。为使综合对比不依赖单一划分，扩展实验使用 5 个随机种子分别构造 5 折交叉验证，共得到 25 个验证折。表 17 给出了各候选模型的平均 RMSE、运行开销和核心结构占用。

表 17: 优化后模型在多随机种子交叉验证中的结果

模型	平均 RMSE	标准差	平均训练时间/s	核心结构/MB	相对原融合提升
原二模型 Ensemble	16.980670	0.086231	21.3901	16.1189	—
调优 ItemCF	17.044970	0.095096	8.3363	18.8308	-0.064301
调优 MF	16.987337	0.081207	18.7919	2.4653	-0.006667
UserCF	17.541538	0.078445	3.2055	10.0352	-0.560868
ItemCF+MF+UserCF 加权融合	16.859807	0.085568	30.3337	31.3313	0.120862

表 17 中的“原二模型 Ensemble”指前文确定的 55% ItemCF 和 45% 矩阵分解融合模型。可以看到，48 维调优 MF 的 RMSE 已接近原二模型融合，说明残差矩阵分解在强正则约束下继续增加维度是有效的；单独 UserCF 的 RMSE 仍然较高，说明仅依赖用户相似性不足以作为主预测方法，但它以较小权重加入融合后可以提供补充信号。最终 ItemCF+MF+UserCF 加权融合的平均 RMSE 降至 16.859807，优于原二模型融合。表中的调优 ItemCF、调优 MF、UserCF 和三模型加权融合训练时间来自当前紧凑存储实现下重新运行的 25 折验证结果；空间占用同样使用当前紧凑数组存储实现重新估算，因此融合模型的核心结构占用约为 31.3313 MB。图 7.10 从准确率、训练时间和核心结构占用三个角度展示了这一权衡。

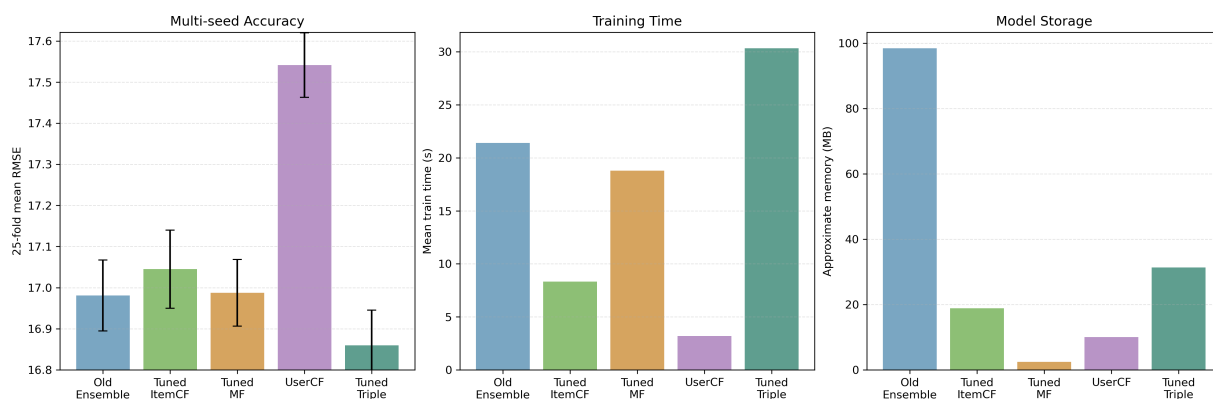


图 7.10: 优化后模型的准确率、训练时间与核心结构占用对比

7.5 多随机种子稳健性验证

由于参数搜索本身可能对某一次交叉验证划分产生适配，仅报告单个 5 折结果仍可能存在偶然性。为检验优化融合的稳定性，实验进一步使用 2024、2025、2026、2027 和 2028 五个随机种子重新构造用户感知的 5 折划分，共得到 25 个验证折。在每一个验证折上，分别计算原二模型融合和优化后三模型融合的 RMSE，并统计配对提升。结果如表 18 所示。

表 18: 原融合与 ItemCF+MF+UserCF 加权融合的多随机种子稳健性比较

模型	25 折平均 RMSE	标准差	最小 RMSE	最大 RMSE	胜出折数
原二模型融合	16.980670	0.086231	16.803201	17.130025	—
ItemCF+MF+UserCF 加权融合	16.859807	0.085568	16.667532	17.031618	25/25

从表 18 可以看出，ItemCF+MF+UserCF 加权融合在 25 个验证折中全部优于原二模型融合，平均 RMSE 降低 0.120862。逐折配对提升的最小值为 0.076936，最大值为 0.154320，说明提升并不是由某一个验证折偶然拉低平均值造成的。图 7.11 进一步展示每个验证折上的 RMSE 曲线和配对提升幅度。

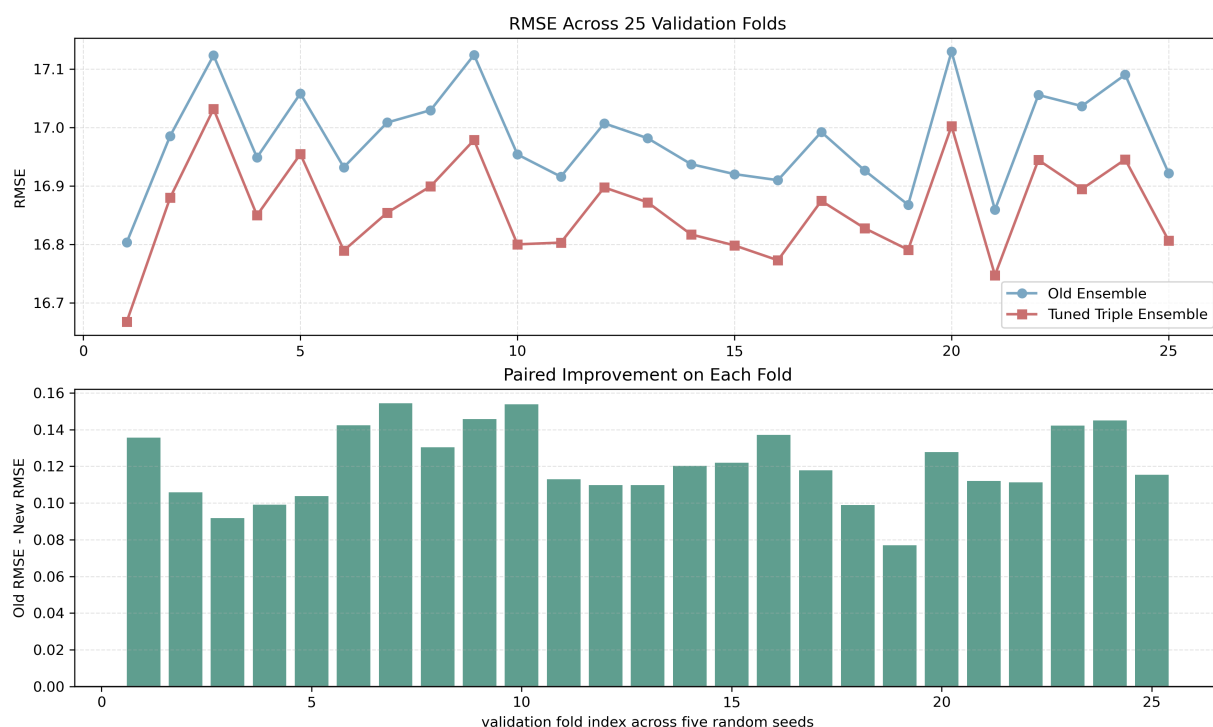


图 7.11: 多随机种子验证中原融合与 ItemCF+MF+UserCF 加权融合的逐折对比

该实验表明, ItemCF+MF+UserCF 加权融合在多个随机划分上均具有稳定优势。需要注意的是, 这一结论仍然只基于 `Train.txt` 内部验证集, 因为 `Test.txt` 不提供真实评分, 无法直接计算测试集 RMSE。因此, 最终模型选择依据是训练集内部交叉验证表现, 而不是测试集反馈。

7.6 最终模型选择与结果文件检查

综合主实验、扩展调参和多随机种子稳健性验证结果, ItemCF、Matrix Factorization 与 UserCF 的加权融合取得最低且更稳定的验证 RMSE, 因此最终预测阶段选择 `OptimizedEnsembleModel` 作为提交模型。该模型使用完整 `Train.txt` 重新训练调优 ItemCF、48 维残差矩阵分解和 UserCF 三个子模型, 并按 0.43 : 0.45 : 0.12 的权重对 `Test.txt` 中的 9982 个用户-物品对输出预测评分。最终预测脚本中保留原自动选择逻辑, 同时允许显式指定优化融合模型。模型构造逻辑如代码片段 39 所示。

任务书要求报告 RMSE、训练时间和空间消耗。为使最终模型选择更加清晰, 表 19 单独列出最终选定模型的核心指标。需要注意的是, `Test.txt` 不包含真实评分, 因此表中的 RMSE 并不是测试集 RMSE, 而是来自 `Train.txt` 内部 5 个随机种子、共 25 个验证折的交叉验证结果; 训练时间、预测时间和核心结构占用则来自最终提交阶段在完整 `Train.txt` 上重新训练一次并预测 `Test.txt` 的实测结果。

表 19: 最终选定模型的核心实验指标

最终模型	平均 RMSE	RMSE 标准差	全量训练时间/s	预测时间/s	核心结构/MB
ItemCF+MF+UserCF	16.859807	0.085568	25.2029	0.9953	36.9539

表 19 中的 ItemCF+MF+UserCF 表示三者按 0.43 : 0.45 : 0.12 进行线性加权融合。其中, RMSE 与 RMSE 标准差来自 5 个随机种子、共 25 个验证折的交叉验证; 训练时间、预测时间和核心结构占用来自最终提交阶段在完整 `Train.txt` 上重新训练一次并预测 `Test.txt` 的实测结果。完整训练阶段

的耗时分解为：ItemCF 约 5.7277 秒，48 维矩阵分解约 18.4284 秒，UserCF 约 1.0467 秒，合计约 25.2029 秒；随后预测 `Test.txt` 用时约 0.9953 秒。最终提交文件还需要满足任务书规定的块状输出格式，因此本节进一步汇总结果文件检查情况，如表 20 所示。

表 20: 最终提交结果文件检查摘要

检查项	结果
最终模型	ItemCF+MF+UserCF 加权融合
待预测用户块数	610
待预测用户-物品对数量	9982
与 <code>Test.txt</code> 用户块顺序是否一致	是
与 <code>Test.txt</code> 物品顺序是否一致	是
预测评分最小值	17.1992
预测评分最大值	100.0000
是否位于 10-100 合法范围内	是

Listing 39: 最终预测阶段选择优化融合模型

```

1 if model_name in {"optimized_ensemble", "optimized"}:
2     return OptimizedEnsembleModel(seed=2026)

```

预测完成后，程序将所有分数截断到训练集评分范围 10 到 100 内，并调用 `write_predictions` 按用户块写出结果文件。最终结果文件生成后，实验脚本进一步对 `final_result.txt` 与 `Test.txt` 进行逐块检查：二者均包含 610 个用户块和 9982 个用户-物品对；每个块的用户编号、物品数量以及块内物品顺序完全一致；每一行预测值均为合法数值。格式检查结果还表明，预测评分最小值为 17.1992，最大值为 100.0000，均位于合法评分区间内。

因此，本实验最终提交结果使用的是 ItemCF、Matrix Factorization 与 UserCF 的加权融合模型。该模型一方面继承了 ItemCF 对相似物品残差的利用能力，另一方面引入矩阵分解对潜在用户-物品交互的低维建模结果，并加入 UserCF 提供用户相似性补充信号。多随机种子验证表明，该融合模型在 25 个验证折中均优于原二模型融合。其不足是需要同时训练并保存三个子模型，因此训练时间和核心结构占用高于原融合模型。

8 总结

本次实验围绕推荐系统评分预测任务展开，目标是在只使用 `Train.txt` 中已知评分的前提下，预测 `Test.txt` 中未知用户-物品对的评分，并按照 `ResultForm.txt` 要求生成结果文件。实验首先对数据集进行了统计分析。训练集中共有 598 个用户、9077 个物品和 90854 条评分记录，评分矩阵密度仅为 1.6738%，同时物品侧存在明显长尾分布；测试集中还包含一定比例的未见用户和未见物品。这些特征决定了模型不能简单依赖完整评分矩阵，而必须同时考虑稀疏性、冷启动回退和输出边界控制。

在算法实现方面，本实验依次实现并比较了 Baseline、ItemCF、UserCF、Matrix Factorization 和 Ensemble 等方法。Baseline 通过全局平均分、用户偏置和物品偏置刻画评分数据中最稳定的系统性因素，并为冷启动样本提供基础回退。ItemCF 在 Baseline 残差空间中计算物品相似度，利用用户在相似物品上的个性化残差修正基础预测。UserCF 从用户相似性角度构造残差修正，为融合模型提供补充信号。Matrix Factorization 使用带偏置项的隐向量模型，通过 SGD 学习用户和物品之间的潜在交互关系。Ensemble 则根据课程中 Hybrid Methods 的思想，将多个推荐器的预测结果进行线性加权融合。

实验结果表明,单一模型之间存在明显的准确率与开销差异。Baseline 的训练时间和核心结构占用最低,但 RMSE 相对较高;ItemCF 通过物品相似邻居获得更低 RMSE,但需要保存大量物品-物品相似关系,因此核心结构占用显著增加;矩阵分解模型更加紧凑,但需要合理控制隐向量维度、学习率和正则化强度;UserCF 单模型效果较弱,但与其他模型存在一定互补性。最终,ItemCF、Matrix Factorization 与 UserCF 的加权融合模型在 5 个随机种子构成的 25 个验证折上取得平均 RMSE 16.859807,优于原二模型融合的 16.980670,并在 25 个验证折中全部胜出。

参数实验进一步说明,推荐算法的效果不仅取决于模型类型,也受到参数选择影响。ItemCF 中的收缩系数能够降低低证据相似关系带来的噪声, `similarity_top_n` 能提高邻居覆盖率但会增加核心结构占用, `block_size` 则主要影响训练时间而不改变预测精度。矩阵分解实验表明,原始训练方式下直接升维会失稳,但在 Baseline 初始化、冻结偏置和较强正则化约束下,48 维残差矩阵分解能进一步降低验证误差。融合实验中,三模型权重 0.43 : 0.45 : 0.12 获得最低验证 RMSE,说明最终模型选择并非凭经验指定,而是由交叉验证和多随机种子稳健性验证共同确定。

综合来看,本实验完成了数据统计、算法实现、参数分析、交叉验证评价、稳健性验证和最终结果生成等要求。最终生成的 `final_result.txt` 与测试集用户块结构保持一致,共包含 610 个用户块和 9982 条预测评分,评分均被限制在训练集评分范围 10 到 100 内。实验的不足在于,本项目只使用了用户-物品评分记录,没有引入物品内容特征、用户属性或时间信息;若后续继续改进,可以考虑更复杂的融合策略、基于验证集的分层误差分析,或结合内容信息缓解长尾物品和冷启动样本的预测困难。